CHAPTER 5 • ML, CORTEX & CERTIFICATION ~15 Min

Student

Can you explain what this chapter is about?

Claude • AI Assistant

This is the payoff chapter. Chapters 1-4 built the platform - architecture, foundations, advanced features, real-world composition patterns. Chapter 5 layers **ML and AI on top** of that platform and closes the loop with **certification prep**.

Section 5.1 (Snowpark ML) brings the classical ML lifecycle inside Snowflake: sklearn-style preprocessing and estimators operating on Snowpark DataFrames, a **Model Registry** for versioning and deployment, and inline SQL scoring via `MODEL!predict(*)` . **Cortex ML Functions** (FORECAST, ANOMALY*DETECTION, CLASSIFICATION, TOP*INSIGHTS) provide ready-made algorithms with a CREATE+CALL pattern - no training code needed.

Snowpark Container Services is the escape hatch for GPU workloads and custom deep learning.

Student

Can you break down the next concept?

Claude • AI Assistant

Section 5.2 (Cortex LLM Functions) exposes large language models as SQL primitives. `COMPLETE` for free-form generation with model choice (llama3.1-70b, mistral-large, claude, and more) and structured message arrays.

`SUMMARIZE` , `TRANSLATE` , `SENTIMENT` , `CLASSIFY_TEXT` , `EXTRACT_ANSWER` for common analytics-shaped tasks - cheaper and faster than general COMPLETE calls. Composable with Streams and Tasks (from Chapter 4) to enrich data in place: new ticket arrives -> stream captures -> task runs Cortex functions -> enriched row lands in analytics.

No data egress, no external API keys, one billing meter.

Section 5.3 (Vector Search + Embeddings + RAG) ties directly back to the transformer material in your Week 5 course. `VECTOR(FLOAT, N)` data type stores embeddings; `EMBED_TEXT_768/1024` produces them via SQL; `VECTOR_COSINE_SIMILARITY` ranks by

semantic similarity; **Cortex Search Service** is the managed hybrid semantic+keyword search endpoint.

Chunk documents, embed each chunk, retrieve top-K, feed to `COMPLETE` with a grounding system prompt - that's the RAG loop, entirely inside Snowflake, no separate vector database.

Student

Can you break down the next concept?

Claude • AI Assistant

Section 5.4 (Cost Optimization) is the discipline that keeps everything above sustainable. Right-size warehouses by testing runtime scaling.

Set `AUTO_SUSPEND = 60` . Attach a **Resource Monitor** to every warehouse with `NOTIFY / SUSPEND / SUSPEND_IMMEDIATE` tiered actions.

Tag objects (`cost_center` , `environment`) and aggregate `METERING_HISTORY` by tag for chargeback. Filter cheaply before calling Cortex LLM functions; cache the results.

Compute is 70-85% of the bill - focus there first.

Section 5.5 (SnowPro Core Prep) maps each of the exam's six domains to specific refcard chapters, offers a 4-week study plan, provides 12 sample questions with explanations, and names the common exam pitfalls (edition-dependent limits, warehouse T-shirt scaling math, micro-partition immutability, stream+Time Travel dependency, owner's-rights defaults).

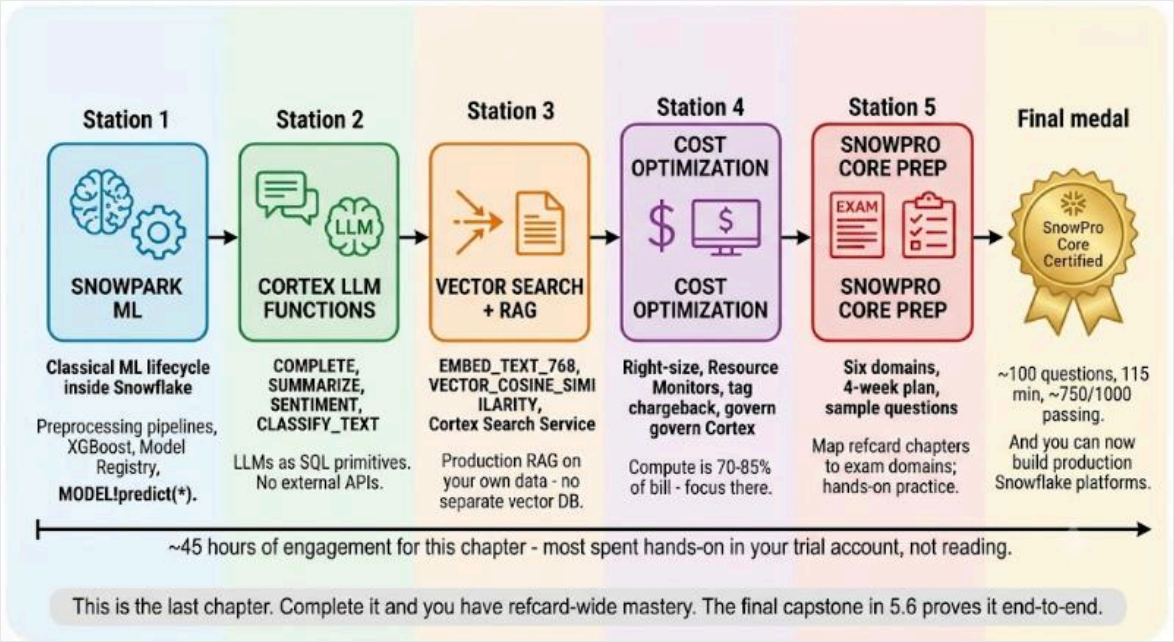
Student

Can you break down the next concept?

Claude • AI Assistant

Section 5.6 (Chapter + Refcard Review) distills the whole refcard into a five-station journey (Foundation -> Foundations -> Advanced -> Real-World Patterns -> ML+Cert) and issues the **final 20-part Mango Mart capstone** - a single project that exercises every mechanism from every chapter. Complete it and you have working practitioner mastery of Snowflake.

By the end of this chapter, you own the ML and AI story on Snowflake, you know how to keep the bill under control, and you are ready to sit the certification exam that recognizes what you have learned across all five chapters.



The Chapter 5 Journey - Five Stations to Mastery, ending in the SnowPro Core Medal

Chapter 5 at a Glance - The Payoff Chapter

Subsection	Title	Core Question Answered	You Can Now...
5.1	Snowpark ML	How do I train and deploy ML models inside Snowflake?	Build sklearn-style pipelines on Snowpark DataFrames, log to Model Registry, score inline in SQL
5.2	Cortex LLM Functions	How do I use LLMs from SQL?	Call COMPLETE, SUMMARIZE, SENTIMENT, CLASSIFY*TEXT, TRANSLATE, EXTRACT*ANSWER directly in queries
5.3	Vector Search, Embeddings, RAG	How do I build a RAG system on Snowflake?	Embed with EMBED*TEXT*768, store as VECTOR, rank with cosine similarity, compose into full RAG loops or use Cortex Search Service
5.4	Cost Optimization	How do I keep the Snowflake bill sane?	Right-size warehouses, set resource monitors, tag objects for chargeback, govern Cortex

Subsection	Title	Core Question Answered	You Can Now...
			spend, monitor via ACCOUNT_USAGE
5.5	SnowPro Core Prep	How do I pass the certification?	Map exam domains to refcard chapters, follow a 4-week study plan, self-check with 12 sample questions
5.6	Chapter + Refcard Review	How does everything fit together?	Complete the 20-part Mango Mart capstone; own the end-to-end vocabulary; know what's beyond this refcard

sql — Chapter 5 preview - Snowpark ML training + Cortex triage + RAG + resource monitor, all in the same account

```sql
-- ML: score every customer using the deployed churn model (5.1)
SELECT customer_id,
       MANGO_MART.ML.CHURN_MODEL!predict(*) AS churn_score
FROM   mango_mart.ml.customer_features_current
ORDER  BY churn_score DESC LIMIT 100;

-- Cortex triage: enrich new tickets with sentiment, category, summary (5.2)
SELECT ticket_id,
       SNOWFLAKE.CORTEX.SENTIMENT(ticket_text)     AS sentiment,
       SNOWFLAKE.CORTEX.CLASSIFY_TEXT(ticket_text,
         ['billing','shipping','product','other']):label::STRING AS category,
       SNOWFLAKE.CORTEX.SUMMARIZE(ticket_text)     AS summary
FROM   mango_mart.raw.new_tickets;

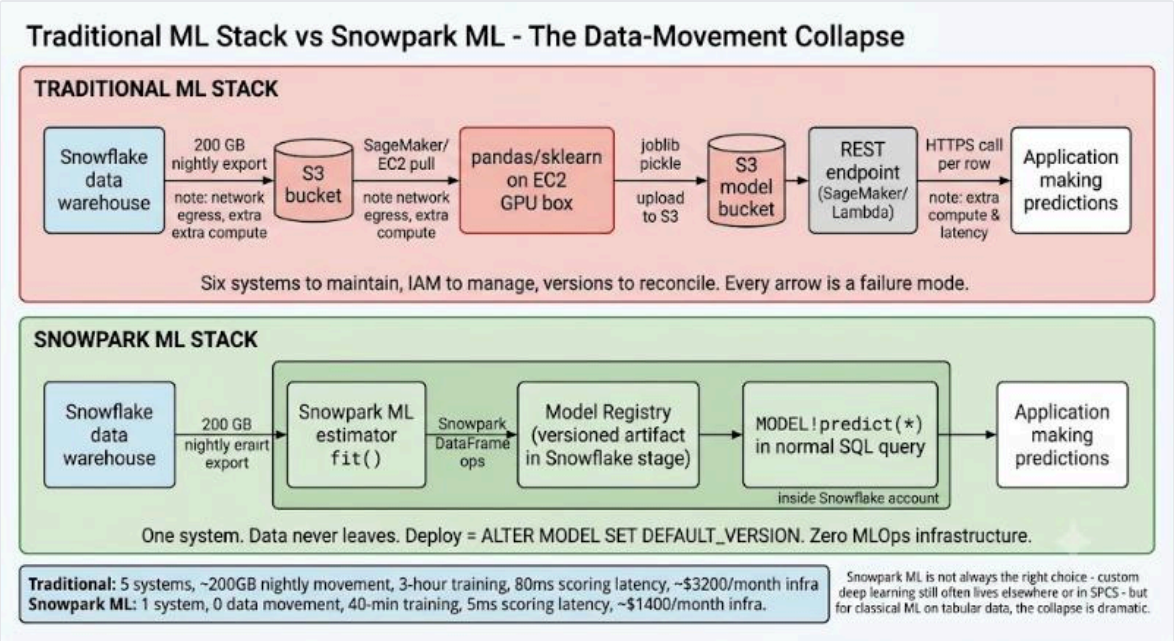
-- RAG: answer a question grounded on the knowledge base (5.3)
SET question = 'What is the return policy for defective mangoes?';
WITH top_chunks AS (
  SELECT chunk_text,
         VECTOR_COSINE_SIMILARITY(
           chunk_embedding,
           SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', $question)) sim
  FROM mango_mart.kb.doc_chunks ORDER BY sim DESC LIMIT 5
)
SELECT SNOWFLAKE.CORTEX.COMPLETE(
  'llama3.1-70b',
  'Answer from context: ' ||
  (SELECT LISTAGG(chunk_text,'\n---\n') FROM top_chunks) ||
```

```
  '\n\nQuestion: ' || $question
) AS answer;

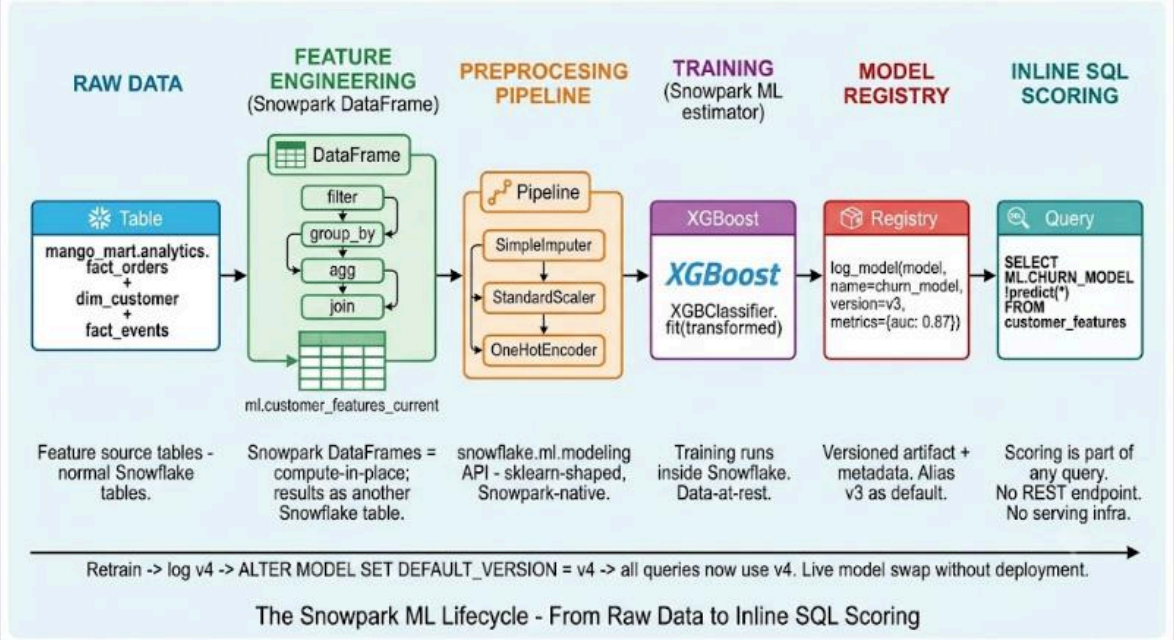
-- Resource monitor: cap the ML warehouse at 500 credits/month (5.4)
CREATE OR REPLACE RESOURCE MONITOR rm_ml
  WITH CREDIT_QUOTA = 500 FREQUENCY = MONTHLY
  TRIGGERS ON 90 PERCENT DO NOTIFY
           ON 100 PERCENT DO SUSPEND
           ON 110 PERCENT DO SUSPEND_IMMEDIATE;
ALTER WAREHOUSE ml_wh SET RESOURCE_MONITOR = rm_ml;
```

SNOWPARK ML - TRAINING AND DEPLOYMENT INSIDE SNOWFLAKE

Claude • AI Assistant



Traditional ML Stack vs Snowpark ML - The Data-Movement Collapse



The Snowpark ML Lifecycle - From Raw Data to Inline SQL Scoring

python — End-to-end Snowpark ML - preprocessing, training, registry, and deployment

```python
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col
from snowflake.ml.modeling.pipeline import Pipeline
from snowflake.ml.modeling.preprocessing import StandardScaler, OneHotEncoder
from snowflake.ml.modeling.impute import SimpleImputer
from snowflake.ml.modeling.xgboost import XGBClassifier
from snowflake.ml.registry import Registry

# --- Session ---
session = Session.builder.configs({
    'user': 'priya', 'private_key_file': '/path/to/key.p8',
    'account': 'xy12345.ap-south-1',
    'warehouse': 'ML_WH', 'database': 'MANGO_MART', 'schema': 'ML',
    'role': 'DATA_SCIENTIST',
}).create()

# --- Load features from Snowflake table ---
features =
session.table('customer_features_current').filter(col('CHURNED').is_not_null(
))
train, test = features.random_split([0.8, 0.2], seed=42)

# --- Preprocessing pipeline (fits and transforms inside Snowflake) ---
numeric_cols     = ['LIFETIME_REVENUE', 'ORDER_COUNT', 'AVG_ORDER_VALUE',
                    'DAYS_SINCE_LAST_ORDER', 'TENURE_DAYS']
categorical_cols = ['CITY', 'ACQUISITION_CHANNEL']
scaled_cols  = [c + '_S'   for c in numeric_cols]
```

```
ohe_cols     = [c + '_OHE' for c in categorical_cols]

prep = Pipeline(steps=[
    ('imp',    SimpleImputer(input_cols=numeric_cols,
output_cols=numeric_cols,
                             strategy='median')),
    ('scale',  StandardScaler(input_cols=numeric_cols,
output_cols=scaled_cols)),
    ('encode', OneHotEncoder(input_cols=categorical_cols,
output_cols=ohe_cols)),
])
train_prepped = prep.fit(train).transform(train)
test_prepped  = prep.transform(test)

# --- Train XGBoost classifier (runs inside Snowflake) ---
feature_cols = scaled_cols + ohe_cols
model = XGBClassifier(
    input_cols   = feature_cols,
    label_cols   = ['CHURNED'],
    output_cols  = ['CHURN_PRED'],
    n_estimators = 200, max_depth = 6, learning_rate = 0.05
)
model.fit(train_prepped)

# --- Evaluate on held-out set ---
from snowflake.ml.modeling.metrics import roc_auc_score
scored = model.predict_proba(test_prepped)
auc = roc_auc_score(df=scored, y_true_col_names='CHURNED',
                    y_score_col_names='PREDICT_PROBA_1')
print(f'Test AUC: {auc:.3f}')

# --- Log to Model Registry ---
registry = Registry(session=session, database_name='MANGO_MART',
schema_name='ML')
reg_model = registry.log_model(
    model             = model,
    model_name        = 'churn_model',
    version_name      = 'v3',
    metrics           = {'auc': float(auc)},
    sample_input_data = train_prepped.limit(50),
    comment           = 'XGBoost, features v2, trained 2026-09-19'
)

# --- Set as production default ---
session.sql("ALTER MODEL mango_mart.ml.churn_model SET DEFAULT_VERSION =
'v3'").collect()
```

```
session.close()
```

sql — Inline SQL scoring and Cortex ML Functions - no Python required at query time

```
-- Score all current customers using the deployed model
SELECT customer_id,
       MANGO_MART.ML.CHURN_MODEL!predict(*) AS churn_score
FROM   mango_mart.ml.customer_features_current
ORDER  BY churn_score DESC
LIMIT  100;

-- List all models in the ML schema
SHOW MODELS IN SCHEMA mango_mart.ml;
SHOW VERSIONS IN MODEL mango_mart.ml.churn_model;

-- Swap production version live (no downtime, no redeploy)
ALTER MODEL mango_mart.ml.churn_model SET DEFAULT_VERSION = 'v4';

-- ================================================
-- Cortex ML Function: 30-day sales forecast (no training code!)
-- ================================================
-- 1. Training data view
CREATE OR REPLACE VIEW mango_mart.ml.v_daily_sales_train AS
SELECT sales_date::DATE       AS sales_date,
       SUM(net_revenue)       AS daily_revenue
FROM   mango_mart.analytics.fact_sales_transaction
WHERE  sales_date >= DATEADD('DAY', -730, CURRENT_DATE)
GROUP  BY sales_date;

-- 2. Create the forecasting model
CREATE OR REPLACE SNOWFLAKE.ML.FORECAST mango_mart.ml.daily_sales_forecast(
  INPUT_DATA        => SYSTEM$REFERENCE('VIEW',
'mango_mart.ml.v_daily_sales_train'),
  TIMESTAMP_COLNAME => 'sales_date',
  TARGET_COLNAME    => 'daily_revenue'
);

-- 3. Produce a 30-day forecast
CALL mango_mart.ml.daily_sales_forecast!FORECAST(FORECASTING_PERIODS => 30);

-- 4. Cortex ANOMALY_DETECTION on live sales
CREATE OR REPLACE SNOWFLAKE.ML.ANOMALY_DETECTION
mango_mart.ml.sales_anomalies(
  INPUT_DATA        => SYSTEM$REFERENCE('VIEW',
```

```
'mango_mart.ml.v_daily_sales_train'),
  TIMESTAMP_COLNAME => 'sales_date',
  TARGET_COLNAME    => 'daily_revenue'
);
CALL mango_mart.ml.sales_anomalies!DETECT_ANOMALIES(
  INPUT_DATA        => SYSTEM$REFERENCE('VIEW',
'mango_mart.ml.v_daily_sales_recent'),
  TIMESTAMP_COLNAME => 'sales_date',
  TARGET_COLNAME    => 'daily_revenue'
);
```

Student

Chapter 4 gave me all the primitives - UDFs, SPs, Tasks. But ML deserves its own treatment. What is Snowpark ML and how does it differ from just writing sklearn code in a Snowpark stored procedure?

Claude • AI Assistant

Snowpark ML is Snowflake's official Python ML library. It's designed to look like scikit-learn but run distributed inside Snowflake on Snowpark DataFrames - so training, scoring, and deployment all happen without data ever leaving the platform.

Around the estimator API sits a **Feature Store** for managed feature definitions, a **Model Registry** for versioning and deployment, and **Snowpark Container Services** for running GPU-based workloads. Together these turn Snowflake from a data warehouse that ML *feeds into* to a platform where the whole ML lifecycle lives.

THE CORE IDEA

Student

What's the mental model?

Claude • AI Assistant

CONCEPT CARD

Traditional ML: **data lives in warehouse -> exported to Python environment -> features engineered in pandas -> trained on GPU box -> pickled -> deployed**

somewhere -> called by an API. Every arrow is a data movement, a versioning problem, or a security boundary.

Snowpark ML collapses all of that:

- **Feature engineering** stays in Snowpark DataFrames (no data leaves).
- **Training** runs distributed inside a Snowflake warehouse (or Container Service for GPU work).
- **Model artifact** lands in the Model Registry as a versioned object.
- **Deployment** is one call to register the model as a Python UDF or a Container Service endpoint.
- **Scoring** is inline in normal SQL (`SELECT model.predict(features) FROM ...`).

The Week 5 deep learning material you built - training transformer-style models locally - moves inside Snowflake with Snowpark ML for classical ML and Cortex/SPCS for LLMs.

FEATURE ENGINEERING WITH SNOWPARK ML

Student

Show me feature engineering.

Claude • AI Assistant

EXAMPLE CARD

Snowpark ML's `snowflake.ml.modeling.preprocessing` and `snowflake.ml.modeling.impute` modules mirror scikit-learn's preprocessing API but operate on Snowpark DataFrames:

python — Code Example

```python
from snowflake.ml.modeling.preprocessing import StandardScaler,
OneHotEncoder
from snowflake.ml.modeling.impute import SimpleImputer
from snowflake.ml.modeling.pipeline import Pipeline

features = session.table('mango_mart.ml.customer_features_current')

numeric_cols     = ['LIFETIME_REVENUE', 'ORDER_COUNT', 'AVG_ORDER_VALUE',
                    'DAYS_SINCE_LAST_ORDER', 'TENURE_DAYS']
categorical_cols = ['CITY', 'ACQUISITION_CHANNEL']

pipeline = Pipeline(steps=[
```

```
    ('imp',    SimpleImputer(input_cols=numeric_cols,
output_cols=numeric_cols,
                             strategy='median')),
    ('scale',  StandardScaler(input_cols=numeric_cols,
                              output_cols=[c + '_SCALED' for c in
numeric_cols])),
    ('encode', OneHotEncoder(input_cols=categorical_cols,
                              output_cols=[c + '_OHE' for c in
categorical_cols]))
])

transformed = pipeline.fit(features).transform(features)
transformed.write.save_as_table('mango_mart.ml.customer_features_transformed
',
                                mode='overwrite')
```

Everything runs inside Snowflake. No pandas, no local memory pressure. The transformed table lands right back in Snowflake for downstream training.

TRAINING A MODEL

Student

Show me the model fit.

Claude · AI Assistant

EXAMPLE CARD

Snowpark ML wraps popular estimators - XGBoost, LightGBM, scikit-learn's classic models - in a Snowpark-native API:

python — Code Example

```
from snowflake.ml.modeling.xgboost import XGBClassifier

feature_cols = [c + '_SCALED' for c in numeric_cols] + [c + '_OHE' for c in
categorical_cols]
label_col    = 'CHURNED'   # 0/1 target

model = XGBClassifier(
    input_cols   = feature_cols,
    label_cols   = [label_col],
    output_cols  = ['CHURN_PRED'],
```

```
    n_estimators = 200,
    max_depth    = 6,
    learning_rate= 0.05
)

model.fit(transformed)      # training runs inside Snowflake

# Score on a held-out set (still all in Snowflake)
scored = model.predict(transformed)
scored.select('CUSTOMER_ID', 'CHURNED', 'CHURN_PRED').show(10)
```

Under the hood, Snowpark ML pushes the fit operation into Snowflake using a stored-procedure job. Data stays put; only the trained artifact is returned to the client.

For large datasets this is the difference between a 90-minute training run (pull data down + fit locally + upload results) and a 6-minute run (fit-in-place).

THE MODEL REGISTRY

Student

How do I keep track of trained models?

Claude · AI Assistant

CONCEPT CARD

The **Model Registry** is a versioned store for trained model artifacts, hosted inside Snowflake. Every model has a name, one or more versions, and rich metadata (metrics, tags, sample data schema, dependencies):

python — Code Example

```
from snowflake.ml.registry import Registry

registry = Registry(session=session, database_name='MANGO_MART',
schema_name='ML')

reg_model = registry.log_model(
    model         = model,
    model_name    = 'churn_model',
    version_name  = 'v3',
    metrics       = {'auc': 0.87, 'precision_at_10': 0.62},
    sample_input_data = transformed.limit(100),
```

```
    comment       = 'XGBoost trained on 2024 Q3+Q4 data, features v2'
)

# Later, load it back and score:
loaded = registry.get_model('churn_model').version('v3')
new_scores = loaded.run(new_customer_features)
```

The Model Registry stores each version as a stage-backed artifact plus rich metadata rows. You can query it like any Snowflake object: `SHOW MODELS`, `DESCRIBE MODEL`, `SHOW VERSIONS IN MODEL churn_model`.

DEPLOYING AS A UDF

Student

How do I make the model callable from SQL?

Claude · AI Assistant

CONCEPT CARD

The registry auto-deploys models as callable functions. You can invoke a model version directly from SQL - no manual UDF wrapping needed:

sql — Code Example

```
-- Score every current customer
SELECT customer_id,
       MANGO_MART.ML.CHURN_MODEL!predict(*)      -- version-aliased default
         AS churn_prediction
FROM   mango_mart.ml.customer_features_current;
```

The `!predict` bang-method calls the model's default version. Update a version alias (`ALTER MODEL churn_model SET DEFAULT_VERSION = 'v4'`) to swap the production model live - all queries pick up the new one on their next execution.

No redeploy, no cache invalidation, no service restart.

CORTEX ML FUNCTIONS - NO TRAINING REQUIRED

Student

Are there pre-built models I can just call?

Claude • AI Assistant

CONCEPT CARD

Cortex ML Functions are ready-to-use algorithms exposed as SQL. You give them a training table and hyperparameters; Snowflake trains a model behind the scenes and gives you a callable object. The main ones:

- **FORECAST** - time-series forecasting (retail demand, capacity planning).
- **ANOMALY_DETECTION** - point/contextual anomaly detection.
- **CLASSIFICATION** - simple binary/multi-class classification.
- **TOP_INSIGHTS** - automatic feature importance / driver analysis.

sql — Code Example

```sql
-- Train a forecasting model on 2 years of daily sales
CREATE OR REPLACE SNOWFLAKE.ML.FORECAST mango_mart.ml.daily_sales_forecast(
  INPUT_DATA => SYSTEM$REFERENCE('VIEW',
'mango_mart.ml.v_daily_sales_train'),
  TIMESTAMP_COLNAME => 'sales_date',
  TARGET_COLNAME    => 'daily_revenue'
);

-- Forecast the next 30 days
CALL mango_mart.ml.daily_sales_forecast!FORECAST(FORECASTING_PERIODS => 30);
```

One CREATE, one CALL, and you have a production-grade forecast. Perfect when you don't need a custom algorithm - you just want a good default.

SNOWPARK CONTAINER SERVICES FOR GPU WORKLOADS

Student

What if I need GPUs for deep learning?

Claude • AI Assistant

CONCEPT CARD

Snowpark Container Services (SPCS) lets you run any containerized workload - including GPU-backed deep learning training and serving - inside Snowflake's compute environment. You provide a Docker image and a compute pool spec; Snowflake runs the container, handles auth, and exposes service endpoints inside your Snowflake network.

SPCS is how you deploy: large transformer models for custom scoring, embedding models Cortex doesn't have, image or video ML workloads, GPU training jobs for deep learning. The container has network access to Snowflake tables via a service-token API, so the container reads Snowflake data, trains or scores, and writes results back - all inside the account boundary.

For most refcard-scope work, classical ML with Snowpark ML on CPU is enough. SPCS is the escape hatch when you outgrow that.

ANALOGY

Snowpark ML is **scikit-learn where the data lives**. The API feels familiar to any data scientist, but the training doesn't pull 100 GB down to a laptop - it runs where the rows already are.

The Model Registry is **git for models**: versioned, tagged, queryable, with the ability to alias 'production' to a specific version. Cortex ML Functions are the **built-in library** for common analytics-style problems (forecasting, anomaly detection) - one CREATE, one CALL.

REAL WORLD APPLICATION

A financial-services analytics team was pulling 200 GB nightly to a SageMaker instance to retrain a fraud model. Migrated to Snowpark ML + Model Registry.

Training moved into Snowflake, took 40 minutes instead of 3 hours, cost about half. The Model Registry replaced their custom Git+S3 versioning setup.

Scoring moved from a REST endpoint to a `SELECT model!predict(*)` call inline in fraud-detection queries. Latency for a single scoring call dropped from 80 ms (REST round-trip) to under 5 ms (in-warehouse).

The SageMaker cluster was decommissioned.

WARNING

Snowpark ML's estimator API looks like scikit-learn but is NOT drop-in compatible. `snowflake.ml.modeling.xgboost.XGBClassifier` takes `input_cols` and `label_cols` (not X, y) and returns a Snowpark DataFrame from `.predict()` (not a numpy array).

Read the actual API docs when porting existing sklearn code - some rewriting is always needed.

COMMON PITFALL / MISTAKE

Trying to fit a Snowpark ML model on a pandas DataFrame. The estimator expects a Snowpark DataFrame that references a Snowflake table.

Convert with `session.create_dataframe(pandas_df)` first (or better, land the data in a Snowflake table before starting - that's the whole point of Snowpark ML).

EXAM TIP CARD

SnowPro Core will test the concepts: (1) Snowpark ML trains models inside Snowflake using Snowpark DataFrames. (2) Model Registry versions models and exposes them as callable objects via `!predict`.

(3) Cortex ML Functions provide FORECAST, ANOMALY*DETECTION, CLASSIFICATION, TOP*INSIGHTS with a CREATE+CALL pattern. (4) SPCS runs containerized/GPU workloads.

Detailed sklearn-style APIs are unlikely in the exam but conceptual knowledge is fair game.

PRACTICE QUIZ

You have a 500 GB customer_features table and want to train an XGBoost churn classifier without moving data out of Snowflake. Best approach?

A) SELECT * from Python Connector, fit sklearn locally, upload predictions

B) Snowpark ML XGBClassifier with input*cols/label*cols, fit on Snowpark DataFrame

C) Cortex FORECAST function

D) Snowpark Container Services with a custom Docker image

▼ Reveal Correct Answer

Answer: B. Snowpark ML is designed for exactly this - distributed training on data-at-rest inside Snowflake. A moves data (defeats the point). C is for time-series, not classification. D is overkill for classical ML on CPU.

EXERCISE

For Mango Mart, build: (a) a Snowpark ML preprocessing pipeline (imputer + scaler + one-hot), (b) an XGBoost churn classifier trained on the transformed features, (c) log to the Model Registry as 'churn*model v1' with AUC in the metadata, (d) score all current customers via `SELECT ML.CHURN*MODEL!predict(*) ...`.

AI DEEP DIVE PROMPT

Ask an AI: *'Design a Snowpark ML pipeline that trains and deploys a customer churn XGBoost model entirely inside Snowflake. Include feature engineering, Model Registry versioning, and*

> *inline SQL scoring. Compare to a traditional SageMaker/MLflow architecture.*'

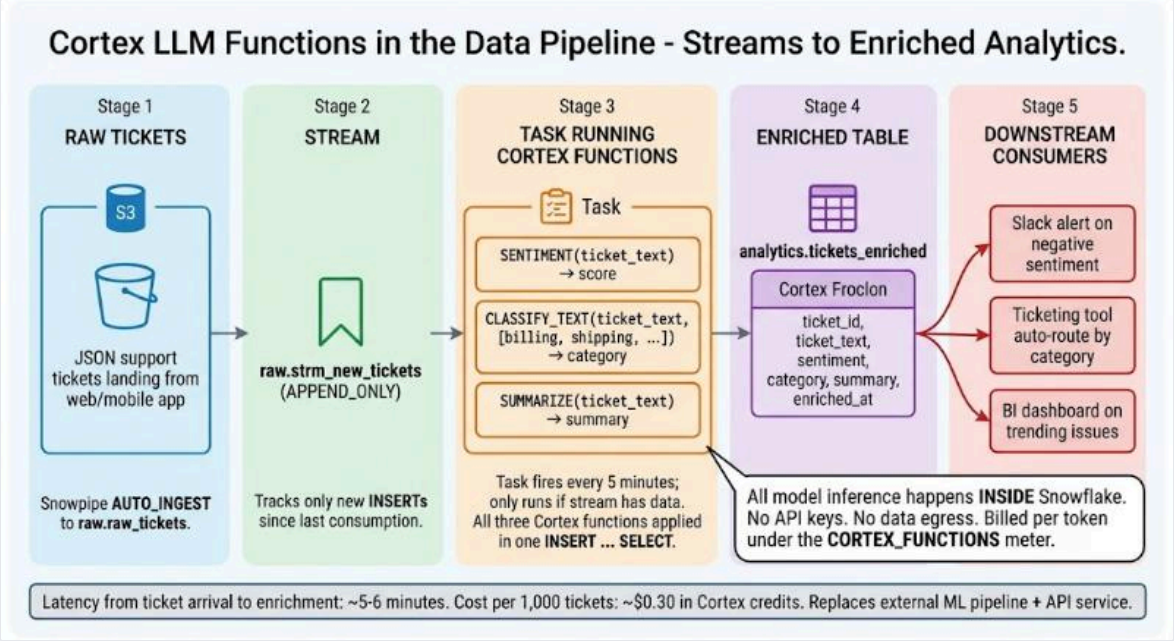
SUMMARY CARD

Snowpark ML brings the ML lifecycle inside Snowflake: sklearn-style estimators trained on Snowpark DataFrames, a **Model Registry** for versioning and one-call deployment, and inline SQL scoring via `MODEL!predict(*)` . **Cortex ML Functions** provide ready-made FORECAST, ANOMALY*DETECTION, CLASSIFICATION, TOP*INSIGHTS via CREATE+CALL.

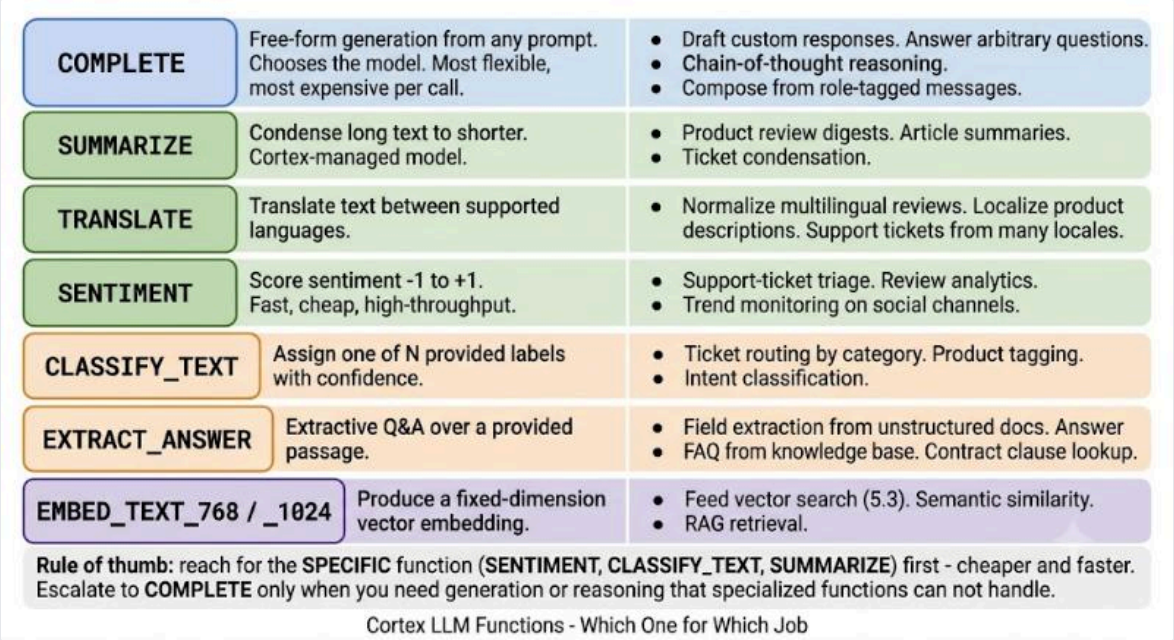
Snowpark Container Services handles GPU/deep-learning workloads. No data movement, no external MLOps stack for classical ML.

CORTEX LLM FUNCTIONS - LARGE LANGUAGE MODELS AS SQL

Claude • AI Assistant



Cortex LLM Functions in the Data Pipeline - Streams to Enriched Analytics



Cortex LLM Functions - Which One for Which Job

sql — Cortex LLM Functions - complete triage pipeline in one Task

```sql
-- Stream on the Snowpipe-fed ticket landing table
CREATE OR REPLACE STREAM mango_mart.raw.strm_new_tickets
  ON TABLE mango_mart.raw.raw_tickets APPEND_ONLY = TRUE;

-- Target: enriched tickets table
CREATE TABLE IF NOT EXISTS mango_mart.analytics.tickets_enriched (
  ticket_id     STRING PRIMARY KEY,
  ticket_text   STRING,
  sentiment     NUMBER(4,3),
  category      STRING,
  summary       STRING,
  enriched_at   TIMESTAMP_LTZ
);

-- Task: run all three Cortex functions every 5 min if stream has data
CREATE OR REPLACE TASK mango_mart.ops.task_triage_tickets
  WAREHOUSE = ml_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('mango_mart.raw.strm_new_tickets')
AS
  INSERT INTO mango_mart.analytics.tickets_enriched
    (ticket_id, ticket_text, sentiment, category, summary, enriched_at)
  SELECT raw_event:ticket_id::STRING,
         raw_event:text::STRING,
         SNOWFLAKE.CORTEX.SENTIMENT(raw_event:text::STRING),
         SNOWFLAKE.CORTEX.CLASSIFY_TEXT(
           raw_event:text::STRING,
```

```
          ['billing','shipping','product_quality','account','other']
        ):label::STRING,
        SNOWFLAKE.CORTEX.SUMMARIZE(raw_event:text::STRING),
        CURRENT_TIMESTAMP
  FROM   mango_mart.raw.strm_new_tickets
  WHERE  METADATA$ACTION = 'INSERT';

ALTER TASK mango_mart.ops.task_triage_tickets RESUME;

-- Query enriched data
SELECT category,
       COUNT(*)              AS tickets,
       ROUND(AVG(sentiment),3) AS avg_sentiment
FROM   mango_mart.analytics.tickets_enriched
WHERE  enriched_at >= DATEADD('HOUR', -24, CURRENT_TIMESTAMP)
GROUP  BY category
ORDER  BY tickets DESC;

-- Monitor Cortex credit consumption
SELECT DATE_TRUNC('DAY', usage_date) AS day,
       ROUND(SUM(credits_used_compute), 2)     AS credits,
       ROUND(SUM(credits_used_compute)*3.0, 2) AS approx_usd
FROM   snowflake.account_usage.metering_history
WHERE  service_type = 'AI_SERVICES'
  AND  usage_date >= DATEADD('DAY', -30, CURRENT_DATE)
GROUP  BY day ORDER BY day;
```

sql — COMPLETE with structured messages, custom drafting, and cost-aware filtering

```
-- Simple COMPLETE with system prompt for consistent tone
SELECT ticket_id,
       SNOWFLAKE.CORTEX.COMPLETE(
         'llama3.1-70b',
         [
           {'role':'system',
            'content':'You are Mango Mart support. Reply warmly in 2-3
sentences.'},
           {'role':'user',    'content': ticket_text}
         ]::VARIANT,
         {'temperature': 0.3, 'max_tokens': 200}::VARIANT
       ) AS suggested_response
FROM   mango_mart.analytics.tickets_enriched
WHERE  category = 'billing'
  AND  sentiment < 0                          -- filter first!
  AND  enriched_at >= DATEADD('HOUR', -1, CURRENT_TIMESTAMP);
```

```
-- Cost pattern: filter cheap, LLM last
-- Step 1: cheap SQL filter reduces 1M reviews -> 12K negative-sentiment ones
-- Step 2: SUMMARIZE (cheap Cortex) on the 12K subset
-- Step 3: COMPLETE (expensive) only on the 500 worst summaries
WITH negative_reviews AS (
  SELECT review_id, review_text
  FROM   mango_mart.analytics.product_reviews
  WHERE  SNOWFLAKE.CORTEX.SENTIMENT(review_text) < -0.5
    AND  review_date >= DATEADD('DAY', -7, CURRENT_DATE)
),
summarized AS (
  SELECT review_id,
         SNOWFLAKE.CORTEX.SUMMARIZE(review_text) AS summary
  FROM   negative_reviews
)
SELECT s.review_id, s.summary,
       SNOWFLAKE.CORTEX.COMPLETE(
         'llama3.1-70b',
         'Analyze this negative review summary and suggest one product '
         'improvement action in one sentence: ' || s.summary
       ) AS action_suggestion
FROM   summarized s
ORDER  BY LENGTH(s.summary) DESC
LIMIT  500;
```

Student

In Week 5 we built transformer architectures from scratch. Now I want to actually USE large language models. What does Snowflake give me?

Claude • AI Assistant

Snowflake Cortex is a suite of managed AI functions exposed as SQL. Instead of setting up your own OpenAI account, managing API keys, and building a scoring pipeline, you call an LLM directly from a SELECT: `SELECT SNOWFLAKE.CORTEX.COMPLETE('llama3.1-70b', prompt) FROM ...` .

Cortex handles auth, rate limiting, model hosting, and billing (per-token). Your data never leaves your Snowflake account - the models run in Snowflake's infrastructure, so you get LLM capabilities with the same governance and security posture as your tables.

This is the direct bridge between the transformer material in your Week 5 curriculum and hands-on production LLM use.

THE COMPLETE FUNCTION

Student

Show me the simplest LLM call.

Claude • AI Assistant

EXAMPLE CARD

`COMPLETE` is a general-purpose text generation function:

sql — Code Example

```sql
SELECT SNOWFLAKE.CORTEX.COMPLETE(
  'llama3.1-70b',
  'You are a retail analyst. In two sentences, explain why mango sales '
  'usually peak in summer.'
) AS explanation;
```

First argument: the model name (one of Cortex's supported models - as of writing, options include `llama3.1-70b`, `llama3.1-405b`, `mistral-large`, `mixtral-8x7b`, and Anthropic Claude models). Second argument: the prompt (or a structured messages array for chat-style interactions).

Return: the model's text response.

This runs against every row of your query. On a support-tickets table with 10,000 rows, you can generate 10,000 summaries in one SQL statement. Costs are per-token - each call bills against your Cortex credit consumption.

STRUCTURED PROMPTING WITH A MESSAGES ARRAY

Student

Can I do multi-turn or role-based prompting?

Claude • AI Assistant

CONCEPT CARD

Yes. Instead of a raw prompt, pass an array of role-tagged messages (system, user, assistant) that mirrors the OpenAI-style chat API:

sql — Code Example

```sql
SELECT SNOWFLAKE.CORTEX.COMPLETE(
  'llama3.1-70b',
  [
    {'role': 'system',    'content': 'You are Mango Mart customer support.
Respond in 2-3 sentences, warm but professional.'},
    {'role': 'user',      'content': ticket_text}
  ]::VARIANT
) AS response
FROM support_tickets
WHERE priority = 'HIGH' AND status = 'OPEN';
```

Also accepts an `options` object for `temperature`, `max_tokens`, `top_p`, etc. Full control from SQL.

SUMMARIZE, TRANSLATE, SENTIMENT - THE ANALYTICS WORKHORSES

Student

What about the more specific functions?

Claude • AI Assistant

EXAMPLE CARD

These are Cortex-tuned functions with narrow purposes:

sql — Code Example

```sql
-- SUMMARIZE: condense long review text
SELECT product_id,
       SNOWFLAKE.CORTEX.SUMMARIZE(review_text) AS review_digest
FROM   product_reviews
WHERE  length(review_text) > 500;

-- TRANSLATE: normalize multilingual customer feedback to English
SELECT ticket_id,
       SNOWFLAKE.CORTEX.TRANSLATE(ticket_text, source_lang, 'en') AS
ticket_en
FROM   support_tickets
```

```
WHERE  source_lang != 'en';

-- SENTIMENT: score on -1 (negative) to +1 (positive)
SELECT product_id,
       AVG(SNOWFLAKE.CORTEX.SENTIMENT(review_text)) AS avg_sentiment
FROM   product_reviews
GROUP  BY product_id
ORDER  BY avg_sentiment ASC LIMIT 20;
```

These are typically cheaper and faster than `COMPLETE` because they use smaller task-specialized models. SENTIMENT in particular is designed for high-throughput analytics on review or ticket streams.

CLASSIFY*TEXT AND EXTRACT*ANSWER

Student

What about classification and Q&A?

Claude • AI Assistant

CONCEPT CARD

Two more analytics-focused functions:

CLASSIFY_TEXT takes a text and a list of candidate labels and returns the best-matching label with a confidence score:

sql — Code Example

```
SELECT ticket_id, ticket_text,
       SNOWFLAKE.CORTEX.CLASSIFY_TEXT(
         ticket_text,
         ['billing', 'shipping', 'product_quality', 'account_access',
'other']
       ):label::STRING AS predicted_category
FROM support_tickets;
```

EXTRACT_ANSWER performs extractive Q&A on a provided text passage - given a question and a context passage, returns the answer if present in the context:

sql — Code Example

```
SELECT product_id,
       SNOWFLAKE.CORTEX.EXTRACT_ANSWER(product_description,
                                       'What is the shelf life?')
         :answer::STRING AS shelf_life
FROM product_catalog;
```

USING LLMS AT SCALE IN A QUERY

Student

What are the gotchas at scale?

Claude • AI Assistant

⚠ WARNING

Cortex LLM functions are **billed per token** and **rate-limited per account**. A `SELECT COMPLETE(...)` over 10 million rows can be expensive and slow (per-call latency ~100ms-2s depending on model). Best practices:

- **Filter aggressively before calling**: apply cheap SQL filters first (`WHERE priority = 'HIGH'`), then only call the LLM on the reduced set.
- **Cache results** in a materialized table: run the LLM once per new row and store the result; subsequent queries read from the cache.
- **Prefer specialized functions** (SUMMARIZE, SENTIMENT, CLASSIFY_TEXT) over COMPLETE for common tasks - they're cheaper.
- **Batch through smaller models first**: use a fast cheap model to filter (`SENTIMENT` to keep only negative reviews), then use a large model (`COMPLETE`) only on the interesting subset.

COMBINING WITH STREAMS AND TASKS

Student

How does this connect back to Chapter 4?

Claude • AI Assistant

CONCEPT CARD

The composition is beautiful. Snowpipe brings support tickets in; a Stream captures new tickets; a Task runs every 5 minutes and applies Cortex functions:

sql — Code Example

```sql
CREATE OR REPLACE TASK ops.task_triage_tickets
  WAREHOUSE = ml_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('raw.strm_new_tickets')
AS
  INSERT INTO analytics.tickets_enriched
  SELECT ticket_id, ticket_text,
         SNOWFLAKE.CORTEX.SENTIMENT(ticket_text)            AS sentiment,
         SNOWFLAKE.CORTEX.CLASSIFY_TEXT(ticket_text,
           ['billing','shipping','product_quality','other']):label::STRING
AS category,
         SNOWFLAKE.CORTEX.SUMMARIZE(ticket_text)            AS summary,
         CURRENT_TIMESTAMP AS enriched_at
  FROM raw.strm_new_tickets
  WHERE METADATA$ACTION = 'INSERT';
```

New tickets are automatically triaged with sentiment, category, and summary within 5 minutes of arrival. No external ML service. No API keys. Pure Snowflake.

THE CORTEX PLAYGROUND

Student

Where do I try this out interactively?

Claude • AI Assistant

Snowsight (the Snowflake UI) has an **AI/ML Studio** with a **Cortex playground** where you can select a model, type prompts, and see responses interactively - great for iterating on system prompts before wiring them into a SQL statement. There is also a chat-style **Cortex Analyst** for asking natural-language questions of your data (which itself compiles to SQL and executes it).

Cortex Analyst is a separate offering worth exploring for BI-adjacent use cases; the LLM functions are the primitive layer beneath it.

ANALOGY

Cortex LLM functions turn **large language models into a SQL function library**. Instead of curl-ing an OpenAI endpoint from a Python script, wiring rate limiting and retries, storing an API key somewhere safe, then joining the results back into your warehouse - you write `SELECT SUMMARIZE(text) FROM reviews` .

The LLM becomes a first-class SQL function in the same way that `LENGTH()` or `LOWER()` are.

REAL WORLD APPLICATION

A support-analytics team at a mid-size retailer was paying an external service to auto-tag and summarize support tickets - about $6,000/month in API fees plus a Python pipeline maintained by a data engineer. Migrated to Cortex CLASSIFY_TEXT + SUMMARIZE in a Task DAG.

Monthly cost dropped to about $900 in Cortex credits, engineer got the pipeline maintenance work back, and the whole pipeline moved inside their Snowflake governance boundary. Bonus: tickets are now enriched within 5 minutes of arrival instead of hourly batches.

WARNING

Cortex LLM functions bill per token, and the billing shows up under a distinct meter (`CORTEX_FUNCTIONS` in ACCOUNT_USAGE). Set resource monitors and cost alerts specifically for Cortex - a runaway `SELECT COMPLETE(...)` on a billion-row table can generate a very large bill quickly.

Use `WAREHOUSE_LOAD_HISTORY` and `METERING_DAILY_HISTORY` to track Cortex consumption.

COMMON PITFALL / MISTAKE

Passing very long text (many thousands of tokens) into COMPLETE and hitting model context limits. Each Cortex model has a documented max token limit; exceeding it errors or truncates silently.

For long documents, chunk them first, summarize each chunk, then summarize the summaries.

EXAM TIP CARD

SnowPro Core will test at concept level: (1) Cortex LLM functions are called from SQL - COMPLETE, SUMMARIZE, TRANSLATE, SENTIMENT, CLASSIFY*TEXT, EXTRACT*ANSWER. (2) Models run inside Snowflake - no data egress.

(3) Per-token billing under a dedicated meter. (4) Composable with Streams, Tasks, and Snowpipe for enriched-in-place pipelines.

PRACTICE QUIZ

You want to auto-triage support tickets: assign a category, score sentiment, and draft a suggested response. Cortex functions in the right order?

A) COMPLETE for category, SENTIMENT for sentiment, TRANSLATE for response

B) CLASSIFY_TEXT for category, SENTIMENT for sentiment, COMPLETE for response

C) SUMMARIZE for all three

D) EXTRACT_ANSWER for all three

▼ Reveal Correct Answer

Answer: B. CLASSIFY_TEXT for label assignment, SENTIMENT for scoring, COMPLETE for open-ended generation. This is the standard triage pipeline shape.

EXERCISE

For Mango Mart, build a Task that runs every 15 minutes, reads new product reviews from a stream, and enriches them with: (a) sentiment score, (b) category (from list ['taste','packaging','shipping','price','other']), (c) a one-line summary. Store enriched results in an analytics table.

AI DEEP DIVE PROMPT

Ask an AI: '*Show a complete Cortex LLM pipeline for triaging customer support tickets: sentiment scoring, category classification, response drafting. Include cost estimation for 100,000 tickets/month and comparison to an OpenAI API-based alternative.*'

SUMMARY CARD

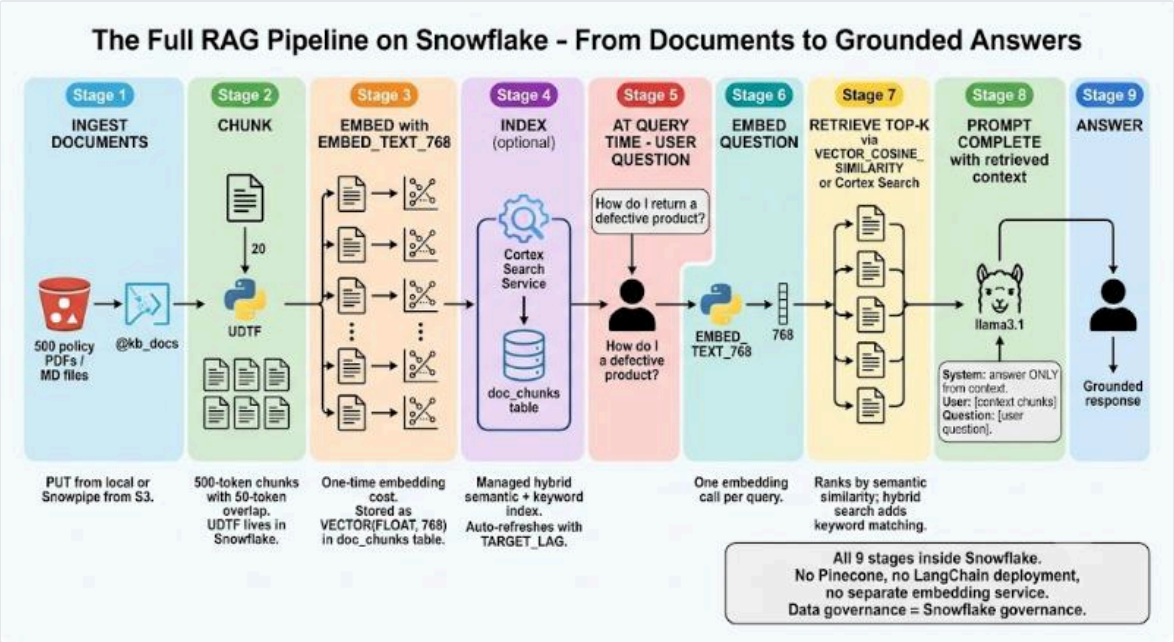
Cortex LLM Functions expose large language models as SQL: **COMPLETE** for general generation, **SUMMARIZE / TRANSLATE / SENTIMENT** for analytics-shaped tasks, **CLASSIFY_TEXT / EXTRACT_ANSWER** for labels and Q&A. Models run inside Snowflake - no data egress, no API keys.

Per-token billing under a dedicated meter. Compose with Streams and Tasks for enriched-in-place pipelines.

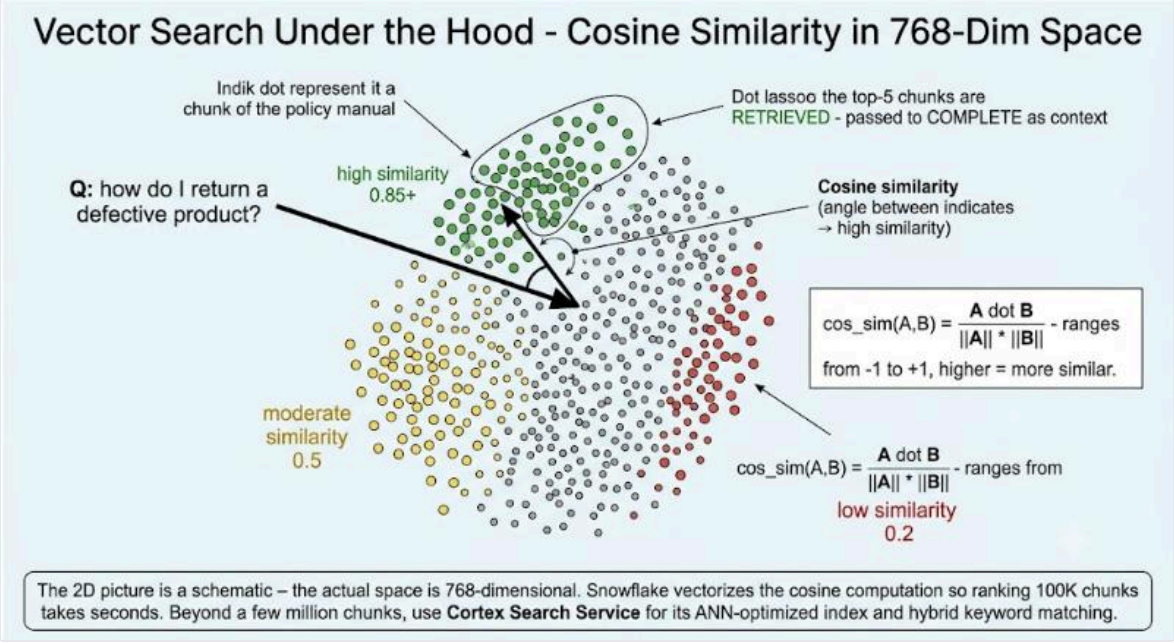
Bridge from the Week 5 transformer material to production LLM use with SQL as the interface.

CORTEX VECTOR SEARCH, EMBEDDINGS, AND RAG

Claude • AI Assistant



The Full RAG Pipeline on Snowflake - From Documents to Grounded Answers



Vector Search Under the Hood - Cosine Similarity in 768-Dim Space

sql — End-to-end RAG in one script - ingest, embed, retrieve, generate

```sql
-- 1. Table for chunked documents with vector embeddings
CREATE OR REPLACE TABLE mango_mart.kb.doc_chunks (
  chunk_id        NUMBER AUTOINCREMENT,
  doc_id          VARCHAR(50),
```

```
  chunk_text      VARCHAR(4000),
  chunk_embedding VECTOR(FLOAT, 768)
);

-- 2. Python UDTF for chunking (from a staged text column)
CREATE OR REPLACE FUNCTION mango_mart.kb.chunk_text_udtf(txt STRING, size
INT, overlap INT)
  RETURNS TABLE (chunk STRING)
  LANGUAGE PYTHON RUNTIME_VERSION = '3.11' HANDLER = 'Chunker'
AS
$$
class Chunker:
    def process(self, txt, size, overlap):
        if not txt: return
        step = max(1, size - overlap)
        for i in range(0, len(txt), step):
            yield (txt[i:i+size],)
$$;

-- 3. Chunk + embed staged docs in one INSERT SELECT
INSERT INTO mango_mart.kb.doc_chunks (doc_id, chunk_text, chunk_embedding)
SELECT d.doc_id,
       t.chunk,
       SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', t.chunk)
FROM   mango_mart.kb.staged_docs d,
  LATERAL TABLE(mango_mart.kb.chunk_text_udtf(d.body, 2000, 200)) t;

-- 4. Sanity check: nearest-neighbour retrieval
SET question = 'How do I return a defective product?';
SELECT chunk_id, doc_id, LEFT(chunk_text, 120) AS preview,
       ROUND(VECTOR_COSINE_SIMILARITY(
               chunk_embedding,
               SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', $question)
             ), 3) AS similarity
FROM   mango_mart.kb.doc_chunks
ORDER  BY similarity DESC LIMIT 5;

-- 5. Full RAG - retrieve then generate in one query
WITH top_chunks AS (
  SELECT chunk_text,
         VECTOR_COSINE_SIMILARITY(
           chunk_embedding,
           SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', $question)
         ) AS sim
  FROM   mango_mart.kb.doc_chunks
  ORDER  BY sim DESC LIMIT 5
```

```
),
context AS (
  SELECT LISTAGG(chunk_text, '\n---\n') WITHIN GROUP (ORDER BY sim DESC) AS
ctx
  FROM   top_chunks
)
SELECT SNOWFLAKE.CORTEX.COMPLETE(
  'llama3.1-70b',
  [
    {'role':'system',
     'content':'You are Mango Mart support. Answer ONLY from the context. '
     'If missing, say you do not have that information.'},
    {'role':'user',
     'content':'Context:\n' || (SELECT ctx FROM context)
                           || '\n\nQuestion: ' || $question}
  ]::VARIANT
) AS grounded_answer;
```

sql — Cortex Search Service - the managed hybrid semantic + keyword search

```
-- 1. Register the search service on the chunks table.
CREATE OR REPLACE CORTEX SEARCH SERVICE mango_mart.kb.doc_search
  ON chunk_text                                -- text column to index
  ATTRIBUTES doc_id                            -- filter/return columns
  WAREHOUSE   = search_wh
  TARGET_LAG  = '5 minutes'                     -- freshness target
  EMBEDDING_MODEL = 'e5-base-v2'
AS (
  SELECT chunk_id, doc_id, chunk_text
  FROM mango_mart.kb.doc_chunks
);

-- 2. Query via SEARCH_PREVIEW (returns JSON).
SELECT PARSE_JSON(
  SNOWFLAKE.CORTEX.SEARCH_PREVIEW(
    'mango_mart.kb.doc_search',
    OBJECT_CONSTRUCT(
      'query',   'return policy for defective mangoes',
      'columns', ARRAY_CONSTRUCT('chunk_text', 'doc_id'),
      'limit',   5
    )::STRING
  )
):results AS results;

-- 3. Monitor service state.
```

```
SHOW CORTEX SEARCH SERVICES IN SCHEMA mango_mart.kb;
DESCRIBE CORTEX SEARCH SERVICE mango_mart.kb.doc_search;

-- 4. REST endpoint for the chatbot UI
-- Chatbot POSTs to:
--
https://<account>.snowflakecomputing.com/api/v2/databases/MANGO_MART/schemas/
KB/cortex-search-services/doc_search:query
-- with body: {"query": "...", "columns": ["chunk_text","doc_id"], "limit":
5}
-- Auth: key-pair or OAuth. Returns JSON with results and scores.
```

Student

Week 5 covered how transformers produce contextual embeddings. Snowflake wraps that as EMBED*TEXT*768. Now how do I actually build a Retrieval-Augmented Generation (RAG) system that answers questions from Mango Mart's internal knowledge base?

Claude • AI Assistant

Perfect - this is where the whole course arc pays off. Snowflake gives you three primitives: the **VECTOR data type** for storing embeddings, **EMBED_TEXT_768 / _1024** Cortex functions to produce embeddings inline, and **VECTOR_COSINE_SIMILARITY** to compare them.

On top of these, the **Cortex Search Service** is a managed hybrid search endpoint that combines semantic (vector) and keyword (lexical) search with a REST API and SQL access. Together these are enough to build a full RAG pipeline where documents are chunked, embedded, and stored in Snowflake, and user questions are answered by retrieving relevant chunks and passing them to `COMPLETE` as context.

THE VECTOR DATA TYPE

Student

What's the storage shape?

Claude • AI Assistant

CONCEPT CARD

Snowflake's `VECTOR(FLOAT, N)` type stores a fixed-dimension array of 32-bit floats. Typical dimensions match embedding-model outputs: 384 (small models), 768 (base BERT-family, `e5-base-v2`), 1024 (larger models), 1536 (OpenAI text-embedding-ada-002).

Store one embedding per row:

sql — Code Example

```sql
CREATE OR REPLACE TABLE mango_mart.kb.doc_chunks (
  chunk_id      NUMBER AUTOINCREMENT,
  doc_id        VARCHAR(50),
  chunk_text    VARCHAR(4000),
  chunk_embedding VECTOR(FLOAT, 768)
);
```

VECTOR columns behave like any other column - storable, queryable, joinable. They participate in normal SQL but with specialized operators (`VECTOR_COSINE_SIMILARITY` , `VECTOR_L2_DISTANCE` , `VECTOR_INNER_PRODUCT`) that compare two vectors and return a scalar.

Snowflake stores vectors compactly and optimizes comparisons via SIMD-style vectorized execution.

GENERATING EMBEDDINGS WITH EMBED*TEXT*768

Student

How do I fill the VECTOR column?

Claude • AI Assistant

EXAMPLE CARD

Use the Cortex embedding function directly in an INSERT SELECT:

sql — Code Example

```sql
INSERT INTO mango_mart.kb.doc_chunks (doc_id, chunk_text, chunk_embedding)
SELECT doc_id,
       chunk_text,
       SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', chunk_text)
  AS chunk_embedding
FROM   mango_mart.kb.staged_chunks;
```

The first argument selects the embedding model (as of writing: `e5-base-v2`, `snowflake-arctic-embed-m`, and a few more). The second is the text to embed.

Cost is per-token embedded, typically cheaper than `COMPLETE` calls. Precompute embeddings once and store them - never re-embed on every query.

CHUNKING - THE CRUCIAL PRE-STEP

Student

How do I turn a big document into chunks?

Claude • AI Assistant

CONCEPT CARD

Embedding models have context limits (typically 512-1024 tokens). Long documents must be **chunked** first - split into pieces small enough to embed and specific enough to retrieve. Standard approach:

- **Fixed-size chunks** (e.g., 500 tokens with 50-token overlap) - simplest, works for uniform text.
- **Semantic chunks** by section/paragraph boundaries - better retrieval quality but requires parsing.
- **Hierarchical chunks** (small chunk + parent chunk) - retrieve small, expand to parent for LLM context.

Chunking can happen inside Snowflake with a Python UDTF (from Chapter 4.3) or inside a Snowpark stored procedure that reads a document from a stage, chunks it, and inserts rows into `doc_chunks`.

THE RETRIEVAL QUERY

Student

How do I do the semantic search?

Claude • AI Assistant

EXAMPLE CARD

Given a user question, embed it and rank chunks by cosine similarity:

sql — Code Example

```sql
SET question = 'How do I return a defective product?';

SELECT chunk_id, doc_id, chunk_text,
       VECTOR_COSINE_SIMILARITY(
         chunk_embedding,
         SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', $question)
       ) AS similarity
FROM   mango_mart.kb.doc_chunks
ORDER  BY similarity DESC
LIMIT  5;
```

Higher cosine similarity means more semantic overlap between the query and the chunk. On 100,000 chunks this executes in seconds - Snowflake vectorizes the comparison efficiently. For hundreds of millions of chunks, Cortex Search Service is a better fit (see below).

THE FULL RAG LOOP

Student

Show me the whole RAG pattern.

Claude • AI Assistant

EXAMPLE CARD

Combine retrieval and generation in one query:

sql — Code Example

```sql
WITH top_chunks AS (
  SELECT chunk_text,
         VECTOR_COSINE_SIMILARITY(
           chunk_embedding,
           SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', $question)
         ) AS sim
  FROM   mango_mart.kb.doc_chunks
  ORDER  BY sim DESC LIMIT 5
),
context AS (
  SELECT LISTAGG(chunk_text, '\n---\n') WITHIN GROUP (ORDER BY sim DESC) AS
ctx
```

```
  FROM top_chunks
)
SELECT SNOWFLAKE.CORTEX.COMPLETE(
  'llama3.1-70b',
  [
    {'role':'system','content':
     'You are Mango Mart support. Answer ONLY using the provided context. '
     'If the answer is not present, say you do not have that information.'},
    {'role':'user','content':
     'Context:\n' || (SELECT ctx FROM context) || '\n\nQuestion: ' ||
$question}
  ]::VARIANT
) AS answer;
```

Retrieve top 5 chunks, concatenate as context, prompt the LLM to answer only from that context. Standard RAG. The whole thing runs in a single Snowflake session; the knowledge base, the retriever, and the generator all live inside the account boundary.

CORTEX SEARCH SERVICE - THE MANAGED LAYER

Student

What if I want it managed?

Claude · AI Assistant

CONCEPT CARD

The **Cortex Search Service** is Snowflake's managed hybrid search offering. You register a table (with a text column and optional filter columns); Cortex takes care of chunking, embedding, indexing, and serving.

Queries can be issued via SQL (`SEARCH_PREVIEW` for testing) or through a REST endpoint suitable for chatbots and apps:

sql — Code Example

```
CREATE OR REPLACE CORTEX SEARCH SERVICE mango_mart.kb.doc_search
  ON chunk_text
  ATTRIBUTES doc_id, category
  WAREHOUSE = search_wh
  TARGET_LAG = '5 minutes'
AS (
  SELECT chunk_id, doc_id, category, chunk_text
```

```
  FROM mango_mart.kb.doc_chunks
);

-- Query via SQL preview
SELECT PARSE_JSON(
  SNOWFLAKE.CORTEX.SEARCH_PREVIEW(
    'mango_mart.kb.doc_search',
    OBJECT_CONSTRUCT(
      'query',   'return policy for defective mangoes',
      'columns', ARRAY_CONSTRUCT('chunk_text', 'doc_id'),
      'limit',   5
    )::STRING
  )
) AS results;
```

The service does **hybrid search** - combining semantic (embedding) and lexical (keyword) matching - which typically outperforms either alone. It auto-refreshes as your source table changes (with the specified TARGET_LAG).

For most production RAG use cases, Cortex Search Service is the right level of abstraction; DIY embeddings + VECTOR*COSINE*SIMILARITY is right when you need custom retrieval logic.

VECTOR SEARCH VS TRADITIONAL SEARCH

Student

When is vector search better than a regular LIKE / SEARCH_OPTIMIZATION query?

Claude • AI Assistant

Vector search excels at **semantic similarity** - matching intent rather than keywords. A query 'how do I get money back for broken goods' should match a document titled 'defective product return policy' even though no exact words overlap.

Keyword search would miss this. That's the vector-search advantage.

But keyword search still wins for exact-match queries (product SKUs, error codes, proper names). This is why **hybrid search** - combining both - typically outperforms either alone. Cortex Search Service does hybrid by default.

TYING BACK TO WEEK 5

Student

How does this connect to the transformer work we did?

Claude • AI Assistant

Everything you learned in Week 5 is exactly what's happening under the hood:

- The **e5-base-v2 embedding model** is a transformer trained with contrastive learning to produce embeddings where semantically-similar sentences land close in vector space.
- The **attention mechanism** you built by hand is what powers both the embedding and the `COMPLETE` (llama3.1) generation.
- The **cosine similarity** ranking is standard nearest-neighbour retrieval - the same principle as the retrieval step you sketched in Week 5's RAG lecture.
- The **prompt-with-context pattern** in `COMPLETE` is exactly what you demonstrated with a mini toy example last week.

You built the plumbing in Python; now you're using industrial versions of the same components via SQL. The concepts don't change; the interface does.

ANALOGY

Vector search in Snowflake is a **semantic index** the way traditional B-tree indexes are lexical indexes. A B-tree finds rows where `column = value` ; a vector index finds rows where `meaning is close to meaning` .

Cortex Search Service is like a **search engine for your Snowflake data** - hybrid, managed, updated automatically, callable from SQL or REST.

REAL WORLD APPLICATION

A B2B software company built an internal support-chatbot RAG system on Snowflake for their 1,200 employees. Documents (product manuals, engineering wiki, past ticket resolutions) were chunked, embedded with EMBED*TEXT*768, and served via a Cortex Search Service.

The chatbot UI called the REST endpoint, retrieved top-5 chunks, and passed them to COMPLETE ('claude-3-5-sonnet') for the final answer. Total build time: 3 weeks.

No separate vector database. No LangChain deployment.

The knowledge base, retrieval, and generation all lived inside Snowflake. Deflection rate on internal tickets: 42% within 3 months.

WARNING

Embedding quality matters. `e5-base-v2` is a solid default but different embedding models have different strengths (English vs multilingual, retrieval vs classification, short queries vs

long documents).

Evaluate on a held-out set of Q&A pairs from your actual domain before committing. Snowflake makes it easy to swap models - just re-run the embedding job with a different first argument.

COMMON PITFALL / MISTAKE

Embedding entire multi-page documents as single vectors. A 20-page document embedded as one vector loses locality - the query matches the document but the LLM gets 20 pages of context, blowing token limits and diluting the answer.

Always chunk first (500-1000 tokens per chunk), embed each chunk, retrieve top-N chunks.

EXAM TIP CARD

SnowPro Core will test at concept level: (1) VECTOR(FLOAT, N) data type for embeddings. (2) EMBED*TEXT*768 / _1024 produce embeddings in SQL.

(3) VECTOR*COSINE*SIMILARITY ranks by semantic similarity. (4) Cortex Search Service is the managed hybrid search offering.

(5) RAG pattern = retrieve top-K chunks + pass to COMPLETE with a grounding system prompt.

PRACTICE QUIZ

You want to build a chatbot answering questions from Mango Mart's 500-page policy manual. What's the correct build order?

A) Send the full manual to COMPLETE with every user question

B) Chunk manual -> EMBED*TEXT*768 each chunk -> at query time embed question -> VECTOR*COSINE*SIMILARITY -> feed top-5 to COMPLETE

C) Use CLASSIFY_TEXT to route questions to relevant pages

D) Store full manual as VECTOR and cosine-search directly

▼ Reveal Correct Answer

Answer: B. Standard RAG. A blows token limits. C classifies but doesn't retrieve passages. D loses locality - embedding entire manual as one vector is meaningless.

EXERCISE

For Mango Mart, build a RAG pipeline over an internal FAQ (100 markdown documents in an internal stage): (a) chunk with a Python UDTF, (b) embed with EMBED*TEXT*768, (c) store

in a table with VECTOR(FLOAT, 768), (d) write a stored procedure that takes a question and returns a grounded answer using COMPLETE, (e) optionally register a Cortex Search Service and query it via REST.

AI DEEP DIVE PROMPT

Ask an AI: '*Design a full RAG system on Snowflake for a customer support chatbot. Include chunking strategy, embedding model choice, vector storage, similarity search, prompt construction, and cost estimation for 1000 queries/day. Compare against a Pinecone + OpenAI stack.*'

SUMMARY CARD

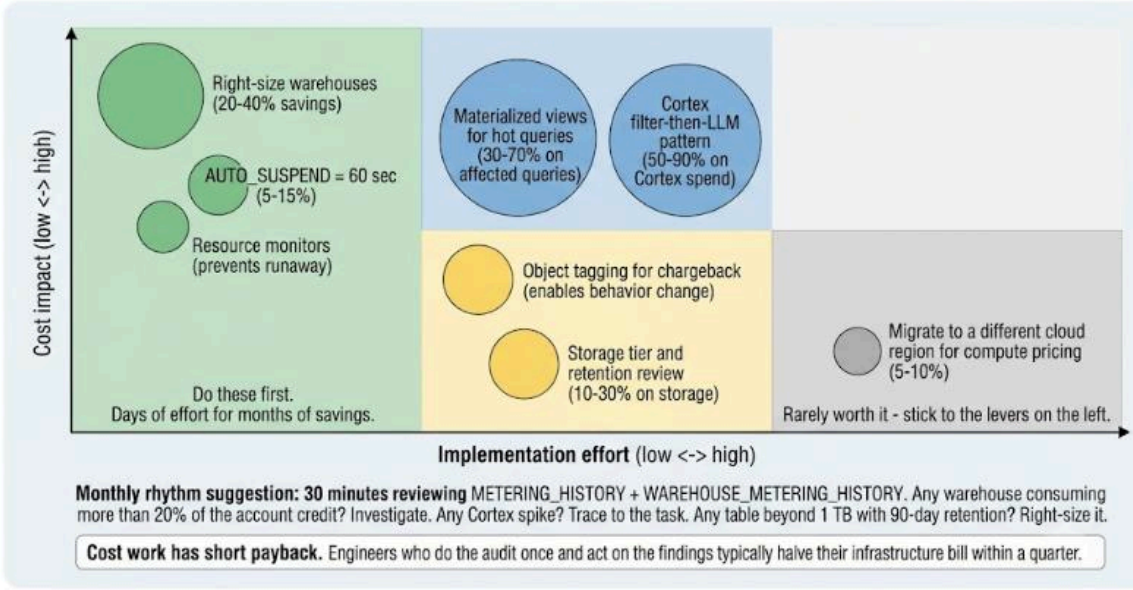
Snowflake makes RAG a first-class SQL pattern: **VECTOR(FLOAT, N)** for storage, **EMBED_TEXT_768/1024** for embedding, **VECTOR_COSINE_SIMILARITY** for retrieval, **COMPLETE** for generation. **Cortex Search Service** is the managed hybrid semantic + keyword search endpoint - queryable via SQL or REST.

Chunk long documents before embedding. Hybrid search typically beats pure vector or pure keyword.

The whole RAG loop lives inside Snowflake - no separate vector database, no LangChain deployment.

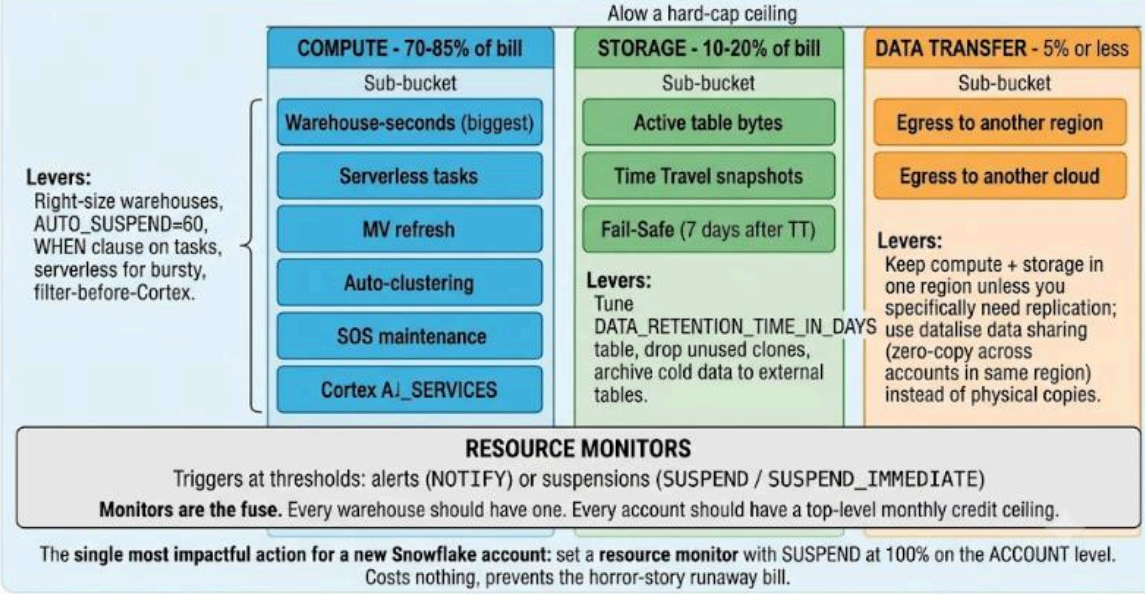
COST OPTIMIZATION - WAREHOUSES, MONITORS, TAGGING, AND CORTEX

Claude • AI Assistant



Cost Optimization Levers Ranked by Impact and Effort

Cost Optimization Levers Ranked by Impact and Effort



The Snowflake Bill - Three Meters and Where Each Optimization Applies.

The Snowflake Bill - Three Meters and Where Each Optimization Applies

sql — Cost dashboard queries - the four every engineer should have bookmarked

```sql
-- 1. Credits per warehouse per day (last 30 days)
SELECT warehouse_name,
       DATE_TRUNC('DAY', start_time) AS day,
       ROUND(SUM(credits_used), 2)               AS credits,
       ROUND(SUM(credits_used) * 3.0, 2)         AS approx_usd
FROM   snowflake.account_usage.warehouse_metering_history
WHERE  start_time >= DATEADD('DAY', -30, CURRENT_TIMESTAMP)
GROUP  BY warehouse_name, day
```

```
ORDER  BY day DESC, credits DESC;

-- 2. Top 20 queries by total credits (last 7 days)
SELECT query_id, warehouse_name, warehouse_size, user_name,
       LEFT(query_text, 80) AS query_preview,
       DATEDIFF('SECOND', start_time, end_time) AS runtime_sec,
       ROUND(credits_used_cloud_services + 0.0, 3) AS credits
FROM   snowflake.account_usage.query_history
WHERE  start_time >= DATEADD('DAY', -7, CURRENT_TIMESTAMP)
ORDER  BY credits DESC LIMIT 20;

-- 3. Storage per table (top 20 largest)
SELECT table_catalog, table_schema, table_name,
       ROUND(active_bytes         / POWER(1024,3), 2) AS active_gb,
       ROUND(time_travel_bytes    / POWER(1024,3), 2) AS tt_gb,
       ROUND(failsafe_bytes       / POWER(1024,3), 2) AS fs_gb,
       ROUND((active_bytes + time_travel_bytes + failsafe_bytes)
              / POWER(1024,3), 2) AS total_gb
FROM   snowflake.account_usage.table_storage_metrics
WHERE  deleted = FALSE
ORDER  BY total_gb DESC LIMIT 20;

-- 4. Cortex AI_SERVICES credits per day
SELECT DATE_TRUNC('DAY', usage_date) AS day,
       ROUND(SUM(credits_used_compute), 2)         AS cortex_credits,
       ROUND(SUM(credits_used_compute) * 3.0, 2)   AS approx_usd
FROM   snowflake.account_usage.metering_history
WHERE  service_type = 'AI_SERVICES'
  AND  usage_date >= DATEADD('DAY', -30, CURRENT_DATE)
GROUP  BY day ORDER BY day DESC;
```

sql — Resource monitors and object tagging for governance

```
-- Create resource monitor with tiered actions.
CREATE OR REPLACE RESOURCE MONITOR rm_etl
  WITH CREDIT_QUOTA  = 500
  FREQUENCY          = MONTHLY
  START_TIMESTAMP    = IMMEDIATELY
  NOTIFY_USERS       = ('priya@mangomart.com', 'ops-alerts@mangomart.com')
  TRIGGERS
    ON 75  PERCENT DO NOTIFY
    ON 90  PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND
    ON 110 PERCENT DO SUSPEND_IMMEDIATE;
```

```
-- Attach to warehouse.
ALTER WAREHOUSE etl_wh SET RESOURCE_MONITOR = rm_etl;

-- Account-level ceiling (the ultimate safety net).
CREATE OR REPLACE RESOURCE MONITOR rm_account_cap
  WITH CREDIT_QUOTA = 5000
  FREQUENCY = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS
    ON 90  PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND
    ON 110 PERCENT DO SUSPEND_IMMEDIATE;
ALTER ACCOUNT SET RESOURCE_MONITOR = rm_account_cap;

-- Object tagging for chargeback.
USE ROLE ACCOUNTADMIN;
CREATE OR REPLACE SCHEMA mango_mart.governance;
CREATE OR REPLACE TAG mango_mart.governance.cost_center;

ALTER WAREHOUSE etl_wh   SET TAG mango_mart.governance.cost_center =
'analytics';
ALTER WAREHOUSE ml_wh    SET TAG mango_mart.governance.cost_center =
'data_science';
ALTER WAREHOUSE bi_wh    SET TAG mango_mart.governance.cost_center =
'finance';
ALTER WAREHOUSE etl_wh   SET TAG mango_mart.governance.environment = 'prod';

-- Chargeback report by cost center.
SELECT tr.tag_value AS cost_center,
       DATE_TRUNC('MONTH', wmh.start_time) AS month,
       ROUND(SUM(wmh.credits_used), 2)         AS credits,
       ROUND(SUM(wmh.credits_used) * 3.0, 2)   AS approx_usd
FROM   snowflake.account_usage.warehouse_metering_history wmh
JOIN   snowflake.account_usage.tag_references tr
         ON tr.object_name = wmh.warehouse_name
        AND tr.tag_name    = 'COST_CENTER'
GROUP  BY cost_center, month
ORDER  BY month DESC, credits DESC;
```

Student

We have all these powerful features. How do I make sure I'm not accidentally spending a fortune?

Claude • AI Assistant

Cost management on Snowflake is a discipline more than a feature. The platform makes it very easy to scale up compute (which is great), which means it's also very easy to overspend (which is not).

Every senior Snowflake practitioner develops a habit of monitoring, right-sizing, and enforcing constraints. This subsection lays out the main levers: **warehouse right-sizing**, **resource monitors**, **object tagging for chargeback**, **Cortex credit governance**, and the essential ACCOUNT_USAGE queries every engineer should have bookmarked.

THE THREE COST BUCKETS

Student

What am I actually being billed for?

Claude • AI Assistant

CONCEPT CARD

Snowflake bills against three separate meters:

1. Compute (credits) - virtual warehouse time (by second, 60-second minimum per resume), serverless task time, MV refresh time, auto-clustering, SOS maintenance, Cortex function invocations. This is usually 70-85% of the total bill.

2. Storage (per TB-month) - active tables, Time Travel snapshots, Fail-Safe. Rate depends on region. Usually 10-20% of the bill.

3. Data transfer (egress) - moving data out of Snowflake to another region or another cloud. Usually small unless you're heavy on cross-cloud replication.

Each meter has its own tuning levers. Compute is where most optimizations live.

WAREHOUSE RIGHT-SIZING

Student

How do I know if my warehouses are the right size?

Claude • AI Assistant

CONCEPT CARD

Snowflake warehouse sizes follow a T-shirt scale (XS, S, M, L, XL, 2XL, 3XL, 4XL, 5XL, 6XL). Each step **doubles** the compute (and doubles the credit rate).

So an XL costs 4x an M and should be able to run a query 4x faster - **if the workload is genuinely parallelizable at that scale**. Otherwise you're paying 4x for maybe 1.5x speedup.

The right-sizing exercise:

1. Pick a representative query. Run it on your current warehouse; note runtime.
2. Rerun on the next-smaller size. Compare runtime.
3. If runtime doubles: your current size is right (linear scaling means you're getting full value).
4. If runtime barely changes: your current size is too big. Drop down; save 50%.
5. If runtime more than doubles: your current size is too small. Go up.

Do this for each workload class - ETL, ad-hoc analytics, BI dashboards. Different workloads want different sizes; don't run everything on one XL.

AUTO_SUSPEND DISCIPLINE

Student

What about idle warehouses?

Claude • AI Assistant

⚠ WARNING

Default `AUTO_SUSPEND = 600` (10 minutes) is too long for most workloads. **Set it to 60 seconds** for most warehouses - the warehouse suspends 60 seconds after the last query, credit consumption stops.

The trade-off: subsequent queries pay the 1-2 second resume delay. For interactive workloads that's fine.

Keep `AUTO_SUSPEND = 600` or higher only for warehouses that need to stay hot for very-latency-sensitive interactive use (e.g., a Tableau server behind a live dashboard where a 2-second warm-up would show up as user-visible lag).

RESOURCE MONITORS - THE GUARDRAILS

Student

How do I prevent runaway spend?

Claude • AI Assistant

CONCEPT CARD

Resource Monitors cap credit consumption. Attach one to a warehouse (or the whole account) with credit limits and actions:

sql — Code Example

```sql
CREATE OR REPLACE RESOURCE MONITOR rm_etl
  WITH CREDIT_QUOTA = 500
  FREQUENCY = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS ON 75 PERCENT DO NOTIFY
           ON 90 PERCENT DO NOTIFY
           ON 100 PERCENT DO SUSPEND
           ON 110 PERCENT DO SUSPEND_IMMEDIATE;

ALTER WAREHOUSE etl_wh SET RESOURCE_MONITOR = rm_etl;
```

At 75%: email the credit contact. At 100%: prevent new queries (running ones finish). At 110%: kill running queries immediately. This is your final safety net - even if a task goes wild, you have a hard credit ceiling.

Best practice: every warehouse should have a resource monitor. Set the quota generously enough to allow normal work but low enough to catch a runaway task within hours.

OBJECT TAGGING FOR CHARGEBACK

Student

How do I track spend by team or project?

Claude • AI Assistant

CONCEPT CARD

Object tags are key-value labels you attach to warehouses, databases, roles, users, and other objects. Tag your warehouses with `team`, `cost_center`, `environment`, and then aggregate `METERING_HISTORY` by tag to build chargeback reports:

sql — Code Example

```sql
CREATE TAG governance.cost_center;
ALTER WAREHOUSE etl_wh   SET TAG governance.cost_center = 'analytics';
ALTER WAREHOUSE ml_wh    SET TAG governance.cost_center = 'data_science';
ALTER WAREHOUSE bi_wh    SET TAG governance.cost_center = 'finance';

-- Chargeback query
SELECT tr.tag_value AS cost_center,
       DATE_TRUNC('MONTH', mh.start_time) AS month,
       ROUND(SUM(mh.credits_used), 2) AS credits,
       ROUND(SUM(mh.credits_used) * 3.0, 2) AS approx_usd
FROM   snowflake.account_usage.metering_history mh
JOIN   snowflake.account_usage.tag_references tr
         ON tr.object_name = mh.name
        AND tr.tag_name = 'COST_CENTER'
GROUP  BY 1, 2 ORDER BY month DESC, credits DESC;
```

Once teams see their own line item, behavior changes. Tagging is as much a management tool as a technical one.

CORTEX CREDIT GOVERNANCE

Student

What about LLM costs?

Claude • AI Assistant

⚠ **WARNING**

Cortex usage bills under a distinct **AI_SERVICES** meter and can grow fast. Standard patterns:

- **Filter cheap, LLM last** - use SQL WHERE to reduce rows before calling COMPLETE.
- **Cache LLM output** - materialize enriched rows in a table; re-use instead of re-inferring.
- **Use specialized functions** - SENTIMENT and CLASSIFY_TEXT are cheaper than COMPLETE.
- **Choose smaller models when they suffice** - `llama3.1-8b` is dramatically cheaper than `llama3.1-405b` for simple tasks.
- **Set a resource monitor** specifically for the warehouse running Cortex tasks.

Monitor Cortex spend with a dedicated ACCOUNT_USAGE query and set a monthly ceiling.

STORAGE OPTIMIZATION

Student

What about storage?

Claude · AI Assistant

CONCEPT CARD

Storage is cheaper than compute per unit but can grow silently. Levers:

- **Drop unused zero-copy clones** - each clone that receives writes accumulates real storage over time.
- **Right-size Time Travel retention** - 90 days on a huge fact table is expensive; 7 or 30 days is often enough.
- **Move cold data to external tables on S3** - archived years can live on cheap S3 storage, queryable but not billed as Snowflake storage.
- **Purge stage files** - Snowpipe leaves loaded files in stages; add REMOVE or lifecycle rules to clean up.

Check `TABLE_STORAGE_METRICS` to see per-table active + time travel + fail safe bytes.

ANALOGY

Cost optimization on Snowflake is like **running a data center where the electricity meter never stops spinning**. Every warehouse-second is billable.

The discipline is: run only what you need, at the smallest size that gets the job done, for the shortest time possible - and put a hard fuse (resource monitor) on every circuit so an accident can't burn down the building.

REAL WORLD APPLICATION

A consumer-tech company noticed their Snowflake bill had grown from 18K/monthto 47K/month over two quarters. Audit found: (1) three warehouses stuck at XL when M would have been fine, (2) two tasks running every 5 minutes with SCHEDULE but no WHEN clause, activating the warehouse on empty runs, (3) a Cortex COMPLETE on every review, uncached.

Fixed all three: bill dropped to $22K/month. Time invested: about 6 engineer-hours.

Cost optimization on Snowflake typically has very short payback periods.

⚠ WARNING

Do not enable auto-scaling multi-cluster warehouses (MIN*CLUSTER*COUNT > 1) unless you actually need concurrent-user scaling. Multi-cluster is a big cost multiplier - each cluster is a full warehouse of the same size, running in parallel.

Useful for BI with hundreds of concurrent users; wasteful for ETL that runs one query at a time.

COMMON PITFALL / MISTAKE

Setting a resource monitor at the account level with only DO NOTIFY (no SUSPEND). Notifications are ignored by tired engineers at 2 AM. Always include DO SUSPEND at 100% so credit limits are enforced, not merely announced.

EXAM TIP CARD

SnowPro Core will test: (1) warehouse right-sizing halves cost when queries do not scale with size. (2) AUTO_SUSPEND controls idle time - default 600 sec, tune to 60 sec for most warehouses.

(3) Resource monitors cap credits with NOTIFY/SUSPEND/SUSPEND*IMMEDIATE actions. (4) Object tags enable chargeback via TAG*REFERENCES + METERING_HISTORY.

(5) Three billing meters: compute credits, storage TB-months, data transfer egress.

PRACTICE QUIZ

Your ETL warehouse is XL and running 5 minutes per hour. You test on L: runtime is 6 minutes. On M: 12 minutes. What's the cost-optimal size?

A) XL - smallest runtime

B) L - almost as fast, half the credit rate

C) M - twice as slow but quarter credit rate

D) 2XL - definitely fastest

▼ Reveal Correct Answer

Answer: B. L runs 20% slower but at half the credit rate = ~40% cost savings for a barely-noticeable delay. M is 2.4x slower at 1/4 rate = only marginally cheaper than L and much slower. XL is clearly oversized (going down one size barely increased runtime).

EXERCISE

For Mango Mart, produce a monthly cost dashboard query that shows: (a) credits per warehouse per week, (b) credits by cost*center tag, (c) Cortex AI*SERVICES credits trend, (d)

largest tables with active + time*travel* + *fail*safe bytes, (e) resource monitor status. Store as a view for the finance team.

AI DEEP DIVE PROMPT

Ask an AI: '*Create a Snowflake cost optimization playbook covering warehouse right-sizing, resource monitors, object tagging for chargeback, Cortex credit governance, and storage optimization. Include a checklist with severity levels and a monthly review cadence.*'

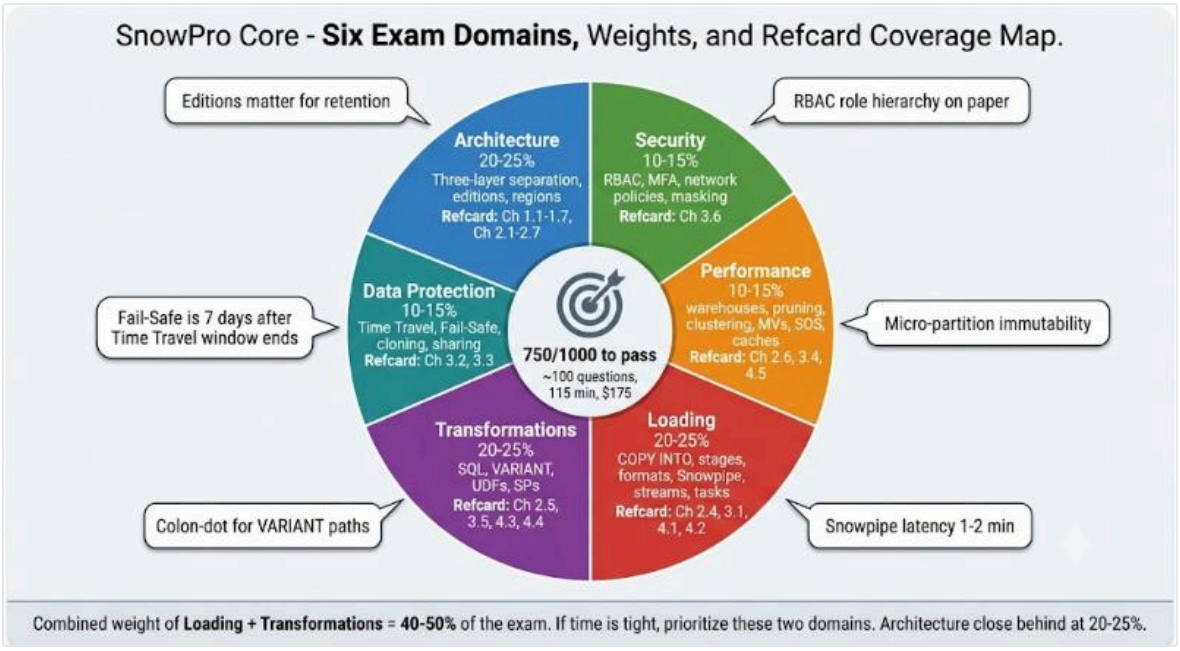
SUMMARY CARD

Snowflake cost optimization is a discipline: **right-size warehouses** by testing runtime scaling, set **AUTO_SUSPEND = 60** as a default, put **Resource Monitors** on every warehouse with NOTIFY + SUSPEND actions, **tag objects** for chargeback aggregation via METERING*HISTORY* + *TAG*REFERENCES, and govern **Cortex credits** by filtering aggressively before LLM calls and caching results. Compute is 70-85% of the bill - focus there first.

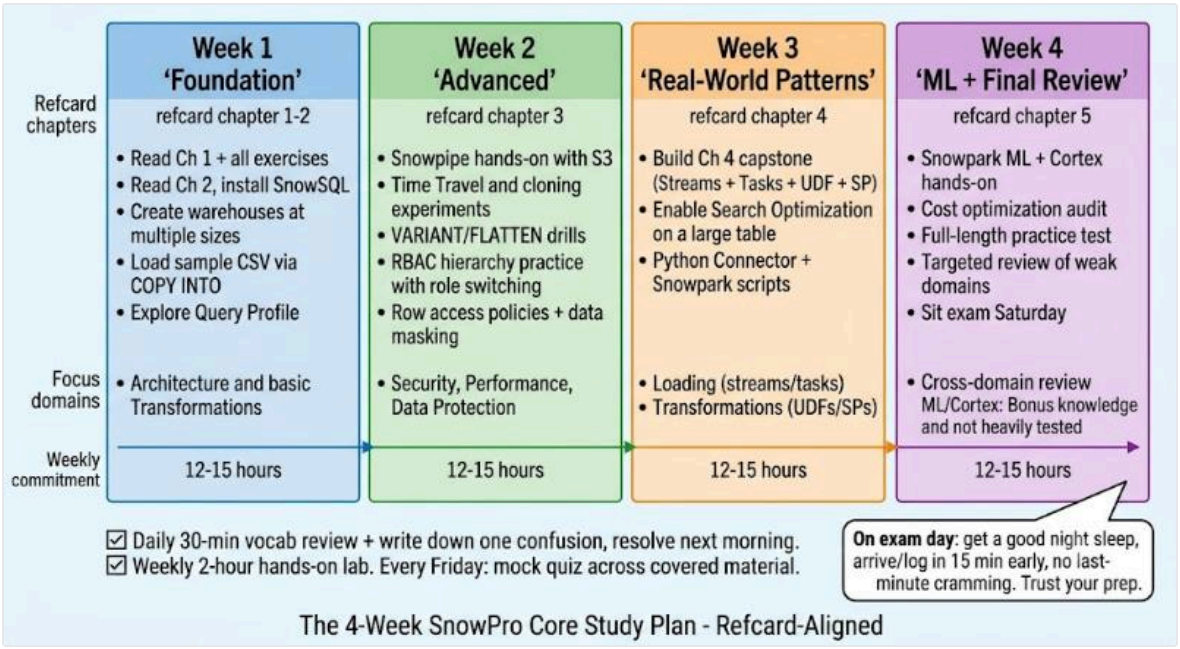
Storage optimization matters too, especially retention and unused clones.

SNOWPRO CORE CERTIFICATION - PREP GUIDE AND STUDY PLAN

Claude • AI Assistant



SnowPro Core - Six Exam Domains, Weights, and Refcard Coverage Map



The 4-Week SnowPro Core Study Plan - Refcard-Aligned

sql — Self-check script - can you answer these SQL questions from memory?

```sql
-- 1. Create a warehouse and set AUTO_SUSPEND to 60 seconds.
-- (Write the CREATE WAREHOUSE statement without looking.)

-- 2. Create a stream on a source table (with Time Travel already enabled).

-- 3. Time-travel query: show a table's state as of 2 hours ago.

-- 4. Create a zero-copy clone of a database.

-- 5. Access a nested field 'address.city' in a VARIANT column.

-- 6. Enable Search Optimization on a column for point lookups.

-- 7. Grant SELECT on all future tables in a schema to a role.

-- 8. Create a task that runs a stored procedure every 5 minutes only if a
stream has data.

-- 9. Create a masking policy that shows email only to a specific role.

-- 10. Query the top 20 queries by credits used in the last 7 days.

-- Suggested answers (attempt yours first, then compare):

-- 1.
CREATE OR REPLACE WAREHOUSE prep_wh
  WITH WAREHOUSE_SIZE='XSMALL' AUTO_SUSPEND=60 AUTO_RESUME=TRUE;
```

```
-- 2.
CREATE OR REPLACE STREAM my_stream ON TABLE my_src APPEND_ONLY=TRUE;

-- 3.
SELECT * FROM my_src AT (OFFSET => -60*60*2);

-- 4.
CREATE DATABASE dev_clone CLONE prod_db;

-- 5.
SELECT profile:address.city::STRING FROM users;

-- 6.
ALTER TABLE fact_orders ADD SEARCH OPTIMIZATION ON EQUALITY(order_id);

-- 7.
GRANT SELECT ON FUTURE TABLES IN SCHEMA raw TO ROLE analyst;

-- 8.
CREATE OR REPLACE TASK t
  WAREHOUSE=etl_wh SCHEDULE='5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('my_stream')
AS CALL my_proc();

-- 9.
CREATE OR REPLACE MASKING POLICY mp_email AS (v STRING)
  RETURNS STRING ->
  CASE WHEN CURRENT_ROLE() = 'PII_ROLE' THEN v ELSE '***@masked' END;

-- 10.
SELECT query_id, LEFT(query_text,80) AS q, credits_used_cloud_services
FROM   snowflake.account_usage.query_history
WHERE  start_time >= DATEADD('DAY',-7,CURRENT_TIMESTAMP)
ORDER  BY credits_used_cloud_services DESC LIMIT 20;
```

Student

I want to sit for the SnowPro Core certification. What's the exam like and how should I use this refcard as prep material?

Claude • AI Assistant

The **SnowPro Core** is Snowflake's foundational certification - one exam, one credential, valid for two years. It's aimed at data engineers, analytics engineers, and admins with **6+ months of hands-on Snowflake experience**.

The exam covers Snowflake's architecture, security, performance, data loading, transformations, and data protection - all at a concept + practical-application level, not a memorize-every-function level. This subsection walks through the exam format, the six domains and their weights, a 4-week study plan aligned to this refcard, and 12 sample questions across the domains.

EXAM FORMAT

Student

What's the actual exam like?

Claude • AI Assistant

CONCEPT CARD

Current format (verify latest at Snowflake's cert page):

- **~100 questions**, mostly single-answer multiple choice, some multi-select and true/false.
- **115 minutes** total.
- **Passing score ~750/1000** (scaled - not a fixed percentage; harder questions weight more).
- **Online-proctored or in-person** at a testing center.
- **English by default**; some other languages available.
- **$175 USD** typical registration fee; retake fees discounted.

Questions test concepts and practical judgment - 'which feature would you use for this scenario' rather than 'what exact syntax'. You will not be asked to write SQL from scratch, but you must recognize correct patterns.

QUESTION STYLE

Student

What do questions look like?

Claude • AI Assistant

EXAMPLE CARD

Typical stem: a scenario in 1-3 sentences, then 4-5 options where 1-2 might look plausible but only one is correct. Examples:

Scenario: 'A team needs to give an external partner read-only access to specific tables without moving data or granting Snowflake account access. Which feature?'

Options: (A) Zero-copy clone shared via S3. (B) Secure Data Sharing with a reader account. (C) Materialized view exported. (D) External function.

Answer: B.

Scenario: 'A stored procedure needs to run privileged operations on behalf of restricted callers. What execution mode?'

Options: (A) EXECUTE AS CALLER. (B) EXECUTE AS OWNER. (C) EXECUTE AS SYSADMIN. (D) EXECUTE AS SECURITYADMIN.

Answer: B - default owner's rights lets restricted callers execute the SP with the owner's privileges.

Notice the pattern: the answer is straightforward if you know the concept, ambiguous if you're guessing. Concept mastery beats memorization.

THE 4-WEEK STUDY PLAN

Student

How should I structure my prep?

Claude • AI Assistant

CONCEPT CARD

Assuming you have this refcard and access to a Snowflake trial account, a solid 4-week plan:

Week 1 - Foundation and Architecture (Refcard Sections 1-2).

- Days 1-2: Read Ch 1 completely. Do all exercises. Focus on the three-layer architecture.
- Days 3-4: Read Ch 2. Practice SnowSQL locally, create warehouses at different sizes.
- Day 5: Load a sample CSV via COPY INTO. Query it. Explore Query Profile.
- Day 6-7: Review + practice questions on Domain 1 and Domain 5 basics.

Week 2 - Advanced Features (Refcard Section 3).

- Days 1-2: Snowpipe hands-on with a real S3 bucket.
- Days 3-4: Time Travel, cloning, VARIANT / FLATTEN exercises.
- Days 5-6: RBAC hierarchy - grant/revoke to functional roles, test with different USE ROLE sessions.
- Day 7: Review + practice questions on Domain 2 (Security), Domain 3 (Performance), Domain 6 (Data Protection).

Week 3 - Real-World Patterns (Refcard Section 4).

- Days 1-2: Build the end-of-Ch-4 capstone - Streams + Tasks + SP + UDF.
- Days 3-4: Search Optimization Service on a large table.
- Days 5-6: Python Connector + Snowpark hands-on.
- Day 7: Review + practice questions on Domain 4 (Loading + Streams + Tasks) and Domain 5 (UDFs + SPs).

Week 4 - ML/Cortex + Final Review (Refcard Section 5).

- Days 1-2: Snowpark ML + Cortex LLM Functions hands-on (may be less emphasized on the exam but understanding is expected).
- Day 3: Cost optimization + resource monitors.
- Day 4: Full-length practice test. Identify weak domains.
- Days 5-6: Targeted review on weak areas.
- Day 7: Rest, light review of vocabulary, sit exam.

Total commitment: roughly 40-60 hours over four weeks (10-15 hours/week). More if you're new to data warehousing.

TWELVE SAMPLE QUESTIONS

Student

Give me a self-check.

Claude • AI Assistant

EXAM TIP CARD

Twelve sample questions across the six domains. Answer each before reading the explanation.

Q1 (Architecture). In Snowflake's three-layer architecture, which layer stores actual table data?

(A) Cloud services (B) Query processing (C) Centralized storage (D) Metadata cache

Answer: C. The centralized storage layer holds micro-partitions on the cloud provider's object storage. Query processing (compute) is stateless and reads from storage.

Q2 (Security). Which is the LEAST-privileged built-in role and typical starting point for standard end-users?

(A) ACCOUNTADMIN (B) SYSADMIN (C) SECURITYADMIN (D) PUBLIC

Answer: D. PUBLIC is the default granted to every user; ACCOUNTADMIN is the most-privileged.

Q3 (Performance). A table is clustered by (date_key). Which query benefits most from clustering-based pruning?

(A) SELECT ... WHERE region = 'SOUTH' (B) SELECT ... WHERE date*key BETWEEN 20241001 AND 20241031 (C) SELECT ... WHERE customer*id = 12345 (D) SELECT COUNT(*) FROM the table

Answer: B. Range predicate on the clustering column enables partition pruning. C is a point lookup - SOS would help there instead.

Q4 (Loading). Snowpipe reduces load latency for continuous ingestion primarily by:

(A) Using larger warehouses (B) Auto-detecting new files and loading them serverlessly within minutes (C) Compressing files at rest (D) Skipping data validation

Answer: B. AUTO_INGEST + event notifications + serverless load = 1-2 minute latency for new files.

Q5 (Transformations). In a VARIANT column holding JSON, how do you access a nested field?

(A) json_extract(col, 'path') (B) col->'path' (C) col:field.subfield (D) col.field.subfield

Answer: C. Snowflake uses colon-and-dot notation for VARIANT path access.

Q6 (Data Protection). Time Travel default retention for standard tables in a Standard edition account is:

(A) 0 days (B) 1 day (C) 7 days (D) 90 days

Answer: B. Default is 1 day; Standard edition max is 1 day; Enterprise+ can extend to 90 days.

Q7 (Streams and CDC). A stream on a source table advances its offset when:

(A) You SELECT from the stream (B) A DML consuming the stream commits successfully (C) The source table is refreshed (D) An external script advances it

Answer: B. SELECTs preview only; successful DML consumption advances the offset.

Q8 (Tasks). Which construct chains child tasks to run after a parent task completes?

(A) SCHEDULE (B) AFTER (C) WHEN (D) TRIGGER

Answer: B. AFTER parent_task creates a DAG dependency; SCHEDULE is for standalone tasks.

Q9 (UDFs). Which UDF language is inline-optimized with zero per-row runtime overhead?

(A) SQL (B) JavaScript (C) Python (D) Java

Answer: A. SQL UDFs are inlined by the query planner into the calling query.

Q10 (Stored Procedures). By default, a Snowflake stored procedure runs with:

(A) Caller's rights (B) Owner's rights (C) Public rights (D) System rights

Answer: B. Owner's rights is the default; use EXECUTE AS CALLER for the alternative.

Q11 (SOS). Search Optimization Service is most effective for:

(A) Range scans on a clustered column (B) Point lookups on a high-cardinality non-clustered column (C) Aggregations across all rows (D) JOIN operations between two tables

Answer: B. SOS accelerates selective predicates finding a small number of rows. Clustering (A), MVs (C), and typical join strategies (D) address the other patterns.

Q12 (Cost). To prevent runaway credit consumption in a warehouse, you attach:

(A) A materialized view (B) A resource monitor with CREDIT_QUOTA and DO SUSPEND triggers (C) A row access policy (D) A network policy

Answer: B. Resource monitors cap credits with tiered actions (NOTIFY at 75%, SUSPEND at 100%, etc.).

COMMON EXAM PITFALLS

Student

What trips people up?

Claude • AI Assistant

⚠ **WARNING**

Watch out for:

- **Editions matter**: Time Travel max is 1 day on Standard, 90 days on Enterprise+. Failsafe is 7 days for permanent tables, 0 for transient/temporary. Questions often key on the edition.
- **Warehouse T-shirt scaling**: each step doubles compute AND credits. XS = 1 cluster of 8 threads; 6XL = 512 nodes of 8 threads each. Do not confuse cluster count (multi-cluster scaling) with warehouse size.

- **Micro-partition immutability**: partitions are never modified in place - updates create new versions. This is why Time Travel works and why cloning is zero-copy.
- **Zero-copy clone divergence**: clones share partitions until either side writes; then the modified partitions diverge but the rest still share. Do not say 'clones are always zero storage cost'.
- **Stream depends on Time Travel**: DATA*RETENTION*TIME*IN*DAYS must be >= 1 for streams to work.
- **SECURE vs standard views**: secure views hide the definition and prevent some optimizations that could leak data. Do not confuse with row access policies.
- **Owner's rights procedures**: default; caller doesn't need underlying privileges. This is the safe-delegation pattern.
- **Snowpipe AUTO_INGEST vs REST API**: AUTO_INGEST uses cloud event notifications; REST API is your service calling `insertFiles` explicitly.
- **MFA + network policies**: standard interview questions on hardening account access.

RESOURCES BEYOND THIS REFCARD

Student

What else should I use?

Claude • AI Assistant

CONCEPT CARD

Additional resources:

- **Snowflake University** (`university.snowflake.com`) - free official courses. The 'Hands-on Essentials' badges (Data Warehouse, Data Applications, Data Lake, Data Engineering, Collaboration) cover exam material well.
- **SnowPro Core Study Guide (official PDF)** - lists every exam objective. Cross-reference against this refcard's coverage.
- **Snowflake documentation** (`docs.snowflake.com`) - authoritative source. Especially the 'Overview and Key Concepts' and 'Security' sections.
- **Practice tests from Whizlabs, ExamTopics, or Udemy** - use for question-style familiarity, not as a primary study source. Some third-party questions are wrong or outdated.
- **A free Snowflake trial account** (30 days, $400 in credits). Do everything hands-on. Reading is not enough.

ANALOGY

The SnowPro Core exam tests **operational fluency**, not encyclopedic recall. Think of it like a driving test: you don't need to name every part of the engine, but you must know when to brake, when to signal, when to yield.

Same here: know when to use a stream vs a task vs a UDF, and why, and you'll pass.

REAL WORLD APPLICATION

A mid-level analytics engineer with 8 months of daily Snowflake use passed the exam after 3 weeks of prep: (1) skim official docs for gaps, (2) complete the hands-on badges at Snowflake University, (3) build one end-to-end capstone project (approximately the Ch 4 capstone in this refcard). No practice tests, no memorization drills.

Score: 850/1000.

COMMON PITFALL / MISTAKE

Studying by watching video courses for 40 hours without touching a Snowflake account. The exam has scenario questions - 'you have symptom X, what fixes it?' - which are near-impossible if you've never actually debugged a query. Balance videos with hands-on ratio at least 1:1.

EXAM TIP CARD

Two hours before the exam: skim the vocabulary lists from each chapter's Chapter Review. Twenty minutes before: close the books - anxiety review hurts more than it helps.

During the exam: answer everything (no penalty for wrong answers), flag hard ones, come back if time permits. Trust your first instinct on scenario questions.

PRACTICE QUIZ

Which chapter of this refcard should you spend the most exam-prep time on if your background is application development rather than data warehousing?

A) Chapter 1 (foundation)

B) Chapter 2 (SnowSQL and loading)

C) Chapter 3 (advanced features)

D) All roughly equal

▼ Reveal Correct Answer

Answer: A + B combined. Application developers usually know SQL but often miss the warehouse and micro-partition mental model, which the exam tests heavily. The advanced features (Ch 3-4) build on that foundation; skimping on Ch 1-2 means Ch 3-4 also underperforms.

EXERCISE

Take a full-length practice exam this weekend and record your score by domain. Identify the two weakest domains, then spend two focused evenings this week re-reading the relevant refcard sections and doing hands-on work in your trial account.

Retake the practice exam next weekend - target at least a 10-point improvement in each weak domain.

AI DEEP DIVE PROMPT

Ask an AI: '*Simulate a 30-question SnowPro Core mock exam covering all six domains with weights matching the official exam. Provide detailed answer explanations that tie back to Snowflake documentation URLs.*'

SUMMARY CARD

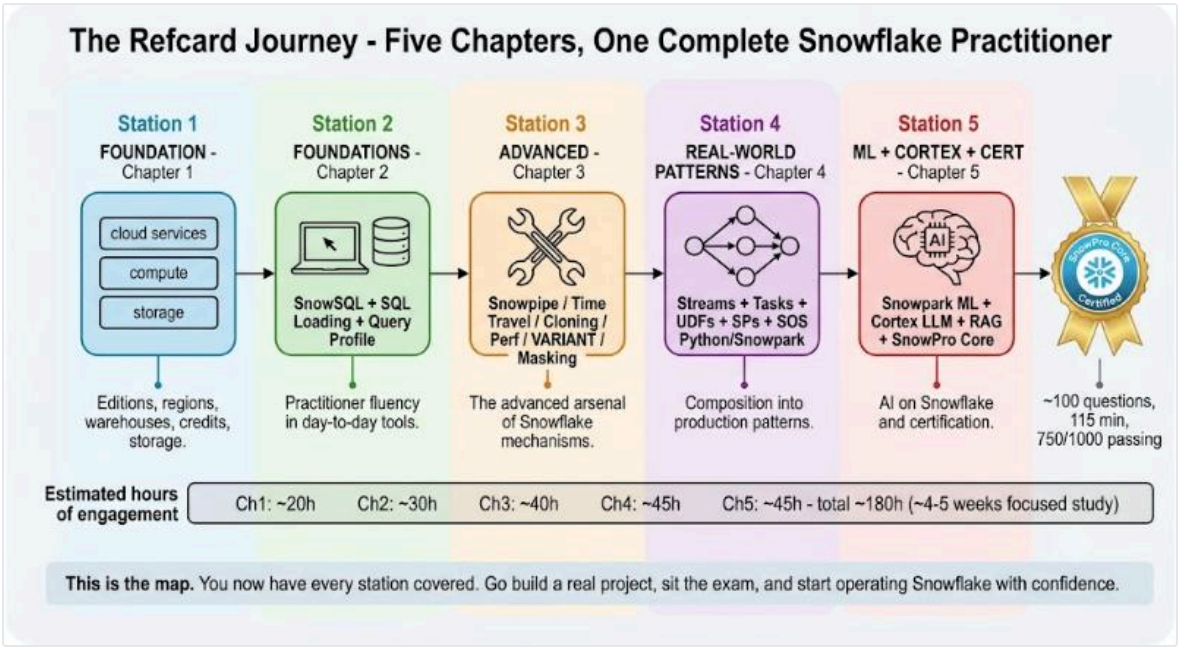
SnowPro Core is Snowflake's foundational certification - one exam, ~100 questions, 115 minutes, ~750/1000 passing. Six domains: Architecture (20-25%), Security (10-15%), Performance (10-15%), Loading (20-25%), Transformations (20-25%), Data Protection (10-15%).

Study by combining this refcard, Snowflake University hands-on badges, and a trial account. Balance conceptual reading with hands-on practice at least 1:1.

Watch out for edition-dependent limits, warehouse scaling math, and stream/Time Travel dependencies. 4 weeks at 10-15 hours/week is a reasonable prep budget for candidates with 6+ months of experience.

CHAPTER 5 REVIEW + REFCARD CONCLUSION

Claude • AI Assistant



The Refcard Journey - Five Chapters, One Complete Snowflake Practitioner

sql — The final capstone - complete Mango Mart platform in one condensed script (each block links back to its refcard section)

```
-- ========== FOUNDATION (Ch 1-2) ==========
USE ROLE SECURITYADMIN;
CREATE ROLE loader; CREATE ROLE transformer; CREATE ROLE analyst;
CREATE ROLE ml_engineer;
GRANT ROLE loader TO ROLE transformer;
GRANT ROLE transformer TO ROLE analyst;
GRANT ROLE analyst TO ROLE ml_engineer;

USE ROLE SYSADMIN;
CREATE DATABASE mango_mart;
CREATE SCHEMA mango_mart.raw;
CREATE SCHEMA mango_mart.analytics;
CREATE SCHEMA mango_mart.ml;
CREATE SCHEMA mango_mart.kb;
CREATE SCHEMA mango_mart.ops;

CREATE WAREHOUSE etl_wh    WITH WAREHOUSE_SIZE='MEDIUM' AUTO_SUSPEND=60
AUTO_RESUME=TRUE;
CREATE WAREHOUSE bi_wh     WITH WAREHOUSE_SIZE='SMALL'  AUTO_SUSPEND=60
AUTO_RESUME=TRUE;
CREATE WAREHOUSE ml_wh     WITH WAREHOUSE_SIZE='LARGE'  AUTO_SUSPEND=60
AUTO_RESUME=TRUE;

-- ========== INGEST (Ch 3.1 + 4.1) ==========
```

```
CREATE OR REPLACE PIPE mango_mart.raw.pipe_orders AUTO_INGEST = TRUE AS
  COPY INTO mango_mart.raw.raw_orders_stream
  FROM @mango_mart.raw.stg_s3 FILE_FORMAT = (TYPE = 'JSON');

CREATE OR REPLACE STREAM mango_mart.raw.strm_new_orders
  ON TABLE mango_mart.raw.raw_orders_stream APPEND_ONLY = TRUE;

-- ========== TRANSFORM (Ch 4.4 + 4.2) ==========
CREATE OR REPLACE PROCEDURE mango_mart.analytics.merge_orders()
  RETURNS VARCHAR LANGUAGE SQL AS
$$ BEGIN
     MERGE INTO mango_mart.analytics.fact_orders t
     USING mango_mart.raw.strm_new_orders s
     ON t.order_id = s.raw_event:order_id::STRING
     WHEN NOT MATCHED THEN INSERT (order_id) VALUES
(s.raw_event:order_id::STRING);
     RETURN 'MERGED ' || SQLROWCOUNT || ' rows'; END; $$;

CREATE OR REPLACE TASK mango_mart.ops.task_merge_orders
  WAREHOUSE = etl_wh SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('mango_mart.raw.strm_new_orders')
AS CALL mango_mart.analytics.merge_orders();
ALTER TASK mango_mart.ops.task_merge_orders RESUME;

-- ========== PERFORMANCE (Ch 3.4 + 4.5) ==========
ALTER TABLE mango_mart.analytics.fact_orders
  CLUSTER BY (paid_at::DATE, customer_id);
ALTER TABLE mango_mart.analytics.fact_orders
  ADD SEARCH OPTIMIZATION ON EQUALITY(order_id, customer_id);

-- ========== SECURITY (Ch 3.6) ==========
CREATE OR REPLACE MASKING POLICY mp_email AS (v STRING)
  RETURNS STRING ->
  CASE WHEN CURRENT_ROLE() IN ('ML_ENGINEER') THEN v ELSE '***@masked' END;
ALTER TABLE mango_mart.retail.dim_customer
  MODIFY COLUMN email SET MASKING POLICY mp_email;

-- ========== ML + AI (Ch 5.1 + 5.2 + 5.3) ==========
-- Snowpark ML training script (run from Python) not shown here.
-- Cortex triage task:
CREATE OR REPLACE TASK mango_mart.ops.task_triage_tickets
  WAREHOUSE = ml_wh SCHEDULE = '15 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('mango_mart.raw.strm_new_tickets')
AS
  INSERT INTO mango_mart.analytics.tickets_enriched
  SELECT ticket_id, ticket_text,
```

```
        SNOWFLAKE.CORTEX.SENTIMENT(ticket_text),
        SNOWFLAKE.CORTEX.CLASSIFY_TEXT(ticket_text,
          ['billing','shipping','product','other']):label::STRING,
        SNOWFLAKE.CORTEX.SUMMARIZE(ticket_text),
        CURRENT_TIMESTAMP
  FROM   mango_mart.raw.strm_new_tickets
  WHERE  METADATA$ACTION = 'INSERT';
ALTER TASK mango_mart.ops.task_triage_tickets RESUME;

-- ========== GOVERNANCE (Ch 5.4) ==========
CREATE OR REPLACE RESOURCE MONITOR rm_account
  WITH CREDIT_QUOTA = 3000 FREQUENCY = MONTHLY START_TIMESTAMP = IMMEDIATELY
  TRIGGERS ON 90 PERCENT DO NOTIFY
           ON 100 PERCENT DO SUSPEND
           ON 110 PERCENT DO SUSPEND_IMMEDIATE;
ALTER ACCOUNT SET RESOURCE_MONITOR = rm_account;

-- Success! Every mechanism from every chapter is exercised above.
-- Now: sit the SnowPro Core exam. Then: go build something real.
```

CHAPTER SUMMARY

Chapter 5 brought the whole picture together. **Snowpark ML** (5.1) puts the classical ML lifecycle inside Snowflake - preprocessing, training, model registry, inline SQL scoring, all without data leaving the platform.

Cortex LLM Functions (5.2) expose large language models as SQL primitives - COMPLETE for generation, SUMMARIZE/TRANSLATE/SENTIMENT/CLASSIFY*TEXT/EXTRACT*ANSWER for common analytics tasks. **Vector Search + Embeddings + RAG** (5.3) added the VECTOR data type, EMBED*TEXT*768/1024, VECTOR*COSINE*SIMILARITY, and the managed Cortex Search Service to build production RAG systems that answer questions from your own data.

Cost Optimization (5.4) gave you the discipline of right-sizing warehouses, setting resource monitors, tagging for chargeback, and governing Cortex spend. **SnowPro Core Prep** (5.5) mapped exam domains to refcard chapters, offered a 4-week study plan, and provided 12 sample questions.

THE REFCARD-WIDE INTEGRATION

You started in Chapter 1 with a mental model of Snowflake's three-layer architecture - separate storage, compute, and cloud services. You built practitioner fluency in Chapter 2 - SnowSQL, roles, COPY INTO, SQL patterns, Query Profile.

Chapter 3 gave you the advanced arsenal - Snowpipe for continuous ingest, Time Travel for historical queries, zero-copy cloning for safe experimentation, clustering + MVs for performance, VARIANT for semi-structured data, RBAC + masking + row access policies for

security. Chapter 4 composed these into production patterns - Streams + Tasks for continuous CDC pipelines, UDFs + Stored Procedures for reusable logic, Search Optimization for point-lookup performance, Python Connector + Snowpark for programmatic access.

Chapter 5 layered ML and AI on top - Snowpark ML for classical ML, Cortex LLM Functions for generative AI, Vector Search for RAG - and taught you to keep the whole thing cost-efficient and to pass the certification exam that validates all of the above.

ONE-SENTENCE REFCARD DISTILLATION

Snowflake is a single-platform data cloud where **storage, compute, and cloud services scale independently**; where **loading, transforming, querying, ML training, LLM inference, and RAG retrieval all happen inside the same governance boundary** with SQL as the universal interface; and where **every advanced feature composes with every other** into production patterns that eliminate most external tooling - all master-able by a working engineer in about a month of focused study.

THE FINAL CAPSTONE - MANGO MART, COMPLETE PLATFORM

> **EXERCISE**
>
> Build the complete Mango Mart data platform end-to-end. Timebox: one week.
>
> **Foundation**:
>
> 1. Create a Snowflake trial account, set up databases (RAW, ANALYTICS, RETAIL, ML, KB, OPS, GOVERNANCE) and schemas.
> 2. Design the functional role hierarchy (LOADER, TRANSFORMER, ANALYST, ML_ENGINEER, ADMIN).
> 3. Create warehouses (LOAD*WH, ETL*WH, BI*WH, ML*WH, SEARCH*WH) at appropriate sizes with AUTO*SUSPEND = 60.
>
> **Ingest**:
>
> 1. Set up Snowpipe from S3 to `raw.raw_orders`, `raw.raw_tickets`, `raw.raw_customer_updates`.
> 2. Streams on each landing table.
>
> **Transform**:
>
> 1. Stored procedures implementing SCD Type 2 merges into `retail.dim_customer`, `retail.dim_product`, `retail.dim_store`.
> 2. Fact table load procedure into `analytics.fact_sales_transaction` from raw orders.
> 3. Task DAG orchestrating all merges - root scheduled at 5 minutes, fan-out to dimension merges, fan-in to fact load.

Performance:

1. Clustering key on fact*sales*transaction by (date*key, store*key).
2. Search Optimization on order*id and customer*id.
3. Materialized views for the two most-queried aggregations.

Security:

1. Masking policies on email and phone in dim_customer, tied to role.
2. Row access policy on fact*sales*transaction limiting store managers to their own store.
3. Resource monitors on every warehouse; account-level ceiling.

ML + AI:

1. Snowpark ML pipeline training a customer churn model; log to Model Registry.
2. Python UDF that scores churn using the registered model - inline in SQL.
3. Task using Cortex CLASSIFY_TEXT + SENTIMENT + SUMMARIZE to triage new tickets.
4. Cortex Search Service on a knowledge-base table, with a RAG query calling COMPLETE for grounded answers.

Programmatic access + observability:

1. Python script using key-pair auth to run daily management queries.
2. Cost dashboard view aggregating credits by warehouse, cost_center tag, and Cortex spend.

If all 20 parts work together, congratulations - you have built a complete production-grade Snowflake data platform. Every mechanism this refcard covered is exercised in the capstone. You are ready for the certification exam and for real work.

VOCABULARY YOU NOW OWN END-TO-END

Chapter 1: three-layer architecture, storage/compute/cloud services, editions (Standard/Enterprise/Business Critical/VPS), warehouse T-shirt sizes, AUTO*SUSPEND, AUTO*RESUME, credit, region, cloud provider.

Chapter 2: SnowSQL, USE ROLE, functional role hierarchy, database/schema/table hierarchy, stages (internal, external, user), COPY INTO, file formats, PUT/GET, CTE, window functions, Query Profile.

Chapter 3: Snowpipe (AUTO*INGEST + REST), micro-partition, Time Travel, AT/BEFORE clause, Fail-Safe, zero-copy clone, clustering key, materialized view, VARIANT, PARSE*JSON, colon-dot path, LATERAL FLATTEN, RBAC, GRANT/REVOKE, masking policy, row access policy, secure view.

Chapter 4: Stream (STANDARD/APPEND*ONLY/INSERT*ONLY), METADATA*ACTION/ISUPDATE/ROW*ID, SCD Type 2, Task (SCHEDULE/AFTER/WHEN), Task

DAG, serverless task, SYSTEM*STREAM*HAS*DATA, TASK*HISTORY, SUSPEND*TASK*AFTER*NUM*FAILURES, SQL/JavaScript/Python UDF, UDTF, external function, SECURE UDF, stored procedure (Snowflake Scripting, Snowpark Python), DECLARE/BEGIN/END/EXCEPTION, EXECUTE IMMEDIATE, owner's/caller's rights, Search Optimization Service (EQUALITY/SUBSTRING/GEO/VARIANT), Python Connector, DB-API 2.0, key-pair auth, write_pandas, Snowpark DataFrame, lazy evaluation.

Chapter 5: Snowpark ML, snowflake.ml.modeling (preprocessing, pipeline, estimator API), Model Registry (log*model, version, DEFAULT*VERSION, !predict), Cortex LLM Functions (COMPLETE, SUMMARIZE, TRANSLATE, SENTIMENT, CLASSIFY*TEXT, EXTRACT*ANSWER, EMBED*TEXT*768/1024), Cortex ML Functions (FORECAST, ANOMALY*DETECTION, CLASSIFICATION), Snowpark Container Services, VECTOR(FLOAT, N), VECTOR*COSINE*SIMILARITY, Cortex Search Service, RAG loop, right-sizing, resource monitor, CREDIT*QUOTA, NOTIFY/SUSPEND/SUSPEND*IMMEDIATE, object tags, TAG*REFERENCES, chargeback, METERING*HISTORY, AI*SERVICES meter, SnowPro Core domains and weights.

WHAT'S BEYOND THIS REFCARD

Deeper study areas we introduced but did not exhaust:

- **Snowpark Container Services** for GPU workloads and custom services.
- **Snowflake Native Apps** for building and distributing apps on the Snowflake Marketplace.
- **Iceberg tables** for external open-format tables with Snowflake compute.
- **Data replication and multi-region failover** (Business Critical edition).
- **Data Clean Rooms** for privacy-preserving joins with partners.
- **Snowflake CLI (`snow`)**, the newer command-line tool complementing SnowSQL.
- **SnowPro Advanced certifications** (Data Engineer, Data Scientist, Architect) for specialization beyond Core.

This refcard equips you as a working Snowflake practitioner. The advanced tracks are for after you've spent 6-12 months in production with Snowflake and want to specialize.

CLOSING NOTE

Snowflake is a big platform and the surface area keeps growing - new Cortex functions, new SPCS options, new ML tooling. The concepts you learned in this refcard - the three-layer architecture, micro-partition immutability, Time Travel, RBAC, streams+tasks - are stable.

New features layer on top of these foundations. Master the foundations and you can absorb whatever Snowflake ships next by reading a doc page or two.

You're now equipped to build, operate, and defend a production Snowflake data platform - and to pass the SnowPro Core certification that recognizes exactly that competence. Go build something.

SUMMARY CARD

Chapter 5 layered ML and AI onto Snowflake's foundational platform: **Snowpark ML** brings classical ML training and deployment inside Snowflake; **Cortex LLM Functions** expose LLMs as SQL primitives; **Vector Search + RAG** enables production question-answering on private data; **Cost Optimization** keeps the bill sane; **SnowPro Core Prep** validates end-to-end competence. Combined with Chapters 1-4, you have complete practitioner mastery of Snowflake - from architecture to advanced features to production patterns to ML/AI to certification.

Go build.